Шинжлэх Ухаан Технологийн Их Сургууль
Мэдээлэл, Холбооны Технологийн Сургууль



F.CSM302 Үйлдлийн систем

БИЕ ДААЛТЫН ТАЙЛАН

Шалгасан багш: Г.Ганбат

Хуваарилалтын код: IV-2-8 (IV өдөр · 2-р цаг · 8-р сэдэв)

Төсөл: Scheduling Algorithms — Operating System Concepts (10th ed.), 5-р бүлэг, Programming Projects

UI дүрслэл: Gantt chart

Гүйцэтгэсэн оюутан: Л.Мөнхжин /B242270094/
B200930051
B221940051
B231940018
B241940025
B241940136

GitHub: https://github.com/Munkhjin12/fcsm302_hw
Онлайн симулятор: https://fcsm302hw.vercel.app

Улаанбаатар хот
2026 он

Гарчиг

1 Удиртгал **5**
1.1 Суурь ойлголт....5
1.2 Асуудлын томьёолол....5
1.3 Ажлын зорилго....5
1.4 Хамрах хүрээ ба таамаглал....6
1.5 Тайлангийн бүтэц....6

2 Сурах бичгийн шаардлага **7**
2.1 Шаардлагын задаргаа....8
2.2 Номын санал болгосон бүтэц....8
2.3 Номд тодорхой бус, бидний шийдсэн асуудлууд....9

3 Онолын үндэс **10**
3.1 Гүйцэтгэлийн үзүүлэлтүүд....10
3.2 Туршилтын өгөгдөл....10
3.3 FCFS (First-Come, First-Served)....11
3.4 SJF (Shortest-Job-First)....11
3.5 Priority scheduling....12
3.6 Round-Robin (RR)....12
3.7 Priority with Round-Robin....13
3.8 Алгоритмуудын онолын хураангуй....13

4 Системийн зохиомж **14**
4.1 Ерөнхий архитектур....14
4.2 C хэрэгжилтийн модулиуд....14
4.3 Өгөгдлийн бүтэц....14
4.3.1 Task бүтэц....14
4.3.2 Холбоост жагсаалт....16
4.4 Нэг интерфэйс — таван хэрэгжилт....16
4.5 Гүйцэтгэлийн урсгал....17
4.6 Симуляторын зохиомж....17

5 Хэрэгжилт **19**
5.1 Хэл, орчин....19
5.2 task.h ба tasks.c: Task ба тооцоолол....19
5.2.1 Task үүсгэх ба атомар tid....19
5.2.2 Яагаад атомар? (Нэмэлт даалгавар 1)....20

5.2.3 report(): дундаж үзүүлэлтүүд (Нэмэлт даалгавар 2) 20
5.3 list.c: холбоост жагсаалт 20
5.4 CPU.c: run() 21
5.5 driver.c: оролтыг шалгах 22
5.6 Алгоритмын файлууд 22
5.6.1 Tie-breaking: яагаад «<» гэж хатуу харьцуулсан бэ? 22
5.6.2 FCFS 23
5.6.3 Round-Robin 23
5.6.4 Priority with Round-Robin 24
5.7 Makefile 24
5.8 Симуляторын хэрэгжилт 24

6 Gantt chart симулятор **25**
6.1 Симуляторын боломжууд 25
6.2 simulate(): 1 ms-ийн алхамтай цөм 25
6.3 Gantt chart-ыг log-оос зурах 27
6.4 «C-ийн гаралт оруулах» — UI ба C-ийн холбоо 27
6.5 Анхаарах зүйл 27

7 Туршилтын арга **29**
7.1 Туршилтын орчин 29
7.2 Туршилтын өгөгдөл 29
7.3 Хүлээгдэх үр дүнг хэрхэн гаргасан бэ? 29
7.4 Шалгалтын төрлүүд 30
7.5 Хязгаарлалт: тестийн хамрах хүрээ 30

8 Үр дүн **32**
8.1 Алгоритм тус бүрийн үр дүн (номын жишээ) 32
8.1.1 FCFS 32
8.1.2 SJF 32
8.1.3 Priority 32
8.1.4 Round-Robin ($q = 10$) 35
8.1.5 Priority + Round-Robin ($q = 10$) 35
8.2 Нэгтгэсэн үр дүн 37
8.3 Тэнцүү priority-ийн туршилт (`schedule_ties.txt`) 37
8.4 Quantum-ын нөлөө 39
8.5 Буруу оролт 40
8.6 UI ба C хэрэгжилтийн тулгалт 40
8.7 Атомар tid-ийн туршилт 42

9 Хэлэлцүүлэг **43**
9.1 Онолын таамаглал ба бодит үр дүн....43
9.2 Алгоритм тус бүрийн дүн шинжилгээ....43
9.3 Starvation-ийн туршилт....44
9.4 Үзүүлэлтүүдийн солилцоо (trade-off)....44
9.5 Хязгаарлалт....45
9.6 Цаашдын сайжруулалт....45
9.7 Ажлаас сурсан зүйл....46

10 Дүгнэлт **47**

A Төслийн мэдээлэл ба тестийн төлөвлөгөө **50**

B Шаардлагын биелэлт **52**

Зургийн жагсаалт

2.1 Номын санал болгосон бүтэц. Саарал хайрцаг нь алгоритм бүрт өөр хэрэгжилттэй хэсэг.....9

4.1 Системийн ерөнхий архитектур. Тасархай сум нь C-ийн гаралтыг симуляторт уншуулах холбоо..... 14
4.2 Ready queue — холбоост жагсаалт. `insert` нь сүүлд, `delete` нь тухайн task-ийг хайж хасна.....16
4.3 `Makefile`: алгоритмын нэг файл + нийтлэг хэсэг = нэг executable. Таван удаа хөрвүүлж таван програм гарна.....16
4.4 `schedule()`-ийн гогцоо. Nonpreemptive алгоритмуудад «remaining > 0» нь хэзээ ч үнэн болдоггүй (task нэг дор дуусна)..... 17

6.1 Симулятор — анхны төлөв. Зүүн талд процессын хүснэгт, алгоритмын сонголт, баруун талд (ажиллуулсны дараа) Gantt chart, метрик гарна.....26
6.2 C-ийн бодит гаралтыг (RR, quantum 4) UI-д ачаалсан байдал, $t = 36$ ms: Gantt, метрик, C-гаралтын 9-р мөр тодорсон.....28

7.1 Симулятор доторх self-test — бүх шалгалт PASS.....31

8.1 FCFS — богино T5 нь хамгийн сүүлд (85 ms хүлээсэн). Бүдэг шугам нь хүлээсэн хугацаа.....33
8.2 SJF — богино task-ууд эхэлж, дундаж waiting хамгийн бага (30)..... 34
8.3 Round-Robin ($q = 10$) — task бүр 10 ms-ээр ээлжилнэ; context switch 10..... 35
8.4 Priority + RR, $t = 35$ алхамд — T5, T1 дууссан; T3 ажиллаж байна; pseudo-code-ийн 8-р мөр тодорсон..... 36
8.5 «Бүгдийг харьцуулах» — 7 алгоритмын дундаж (симулятор).....38
8.6 Quantum-ын нөлөө (RR, номын өгөгдөл). Зүүн: waiting ба response; баруун: context switch (логарифм тэнхлэг).....40
8.7 Буруу оролт (quantum = 0) — ойлгомжтой алдааны мэдээлэл, ажиллуулалт хийгдэхгүй..... 41

Хүснэгтийн жагсаалт

2.1 Номын шаардлага ба бидний хэрэгжүүлэлт 8

3.1 Номын жишээ өгөгдөл (`tests/schedule.txt`) 10
3.2 Алгоритмуудын шинж чанарын харьцуулалт 13

4.1 Хэрэгжилтийн файлууд ба тэдгээрийн үүрэг 15
4.2 `Task` бүтцийн талбарууд 15

5.1 Оролтын шалгалт ба алдааны хариу 22

7.1 `make test`-ийн 18 шалгалт 30

8.1 Алгоритм тус бүрийн дундаж үзүүлэлт (ms) ба context switch-ийн тоо 37
8.2 Task бүрийн waiting time (ms) 37
8.3 Task бүрийн анх CPU авсан цаг (response) ба дууссан цаг (completion), ms . . . 38
8.4 Quantum-аас хамаарсан дундаж үзүүлэлт (ms) ба context switch 39
8.5 UI-ийн тооцсон дундаж ба C-ийн хэвлэсэн дундажийн тулгалт 41

9.1 Онолын таамаглал ба манай үр дүн 43
9.2 L task-ийн хүлээлт: FCFS ба Priority 44

A.1 Төслийн ерөнхий мэдээлэл 50
A.2 Тестийн төлөвлөгөө 51

B.1 Шаардлагын биелэлт ба нотолгоо 52

Төслийн тайлбар толь

Нэр томъёо	Тайлбар
Arrival time	Процесс ready queue-д ирсэн цаг. Номын нөхцөлд бүх task $t = 0$ үед ирнэ.
Burst (CPU burst)	Task-ийн CPU дээр ажиллах шаардлагатай нийт хугацаа (ms).
Context switch	CPU-г нэг task-аас нөгөөд шилжүүлэх үйлдэл. Энэ ажилд зардлыг 0 ms гэж үзсэн.
Convoy effect	Урт task урд байх үед богино task-ууд удаан хүлээх үзэгдэл (FCFS-д ажиглагдана).
FCFS	First-Come, First-Served — ирсэн дарааллаар үйлчлэх алгоритм.
Gantt chart	CPU дээр аль цагт аль task ажилласныг цагийн тэнхлэгт дүрсэлсэн диаграм.
Idle	CPU сул байх үе (ямар ч процесс ирээгүй). Симуляторт судалтай блокоор харагдана.
Priority	Task-ийн эрэмбэ (1–10). Их тоо = өндөр priority.
Priority + RR	Priority-оор сонгож, тэнцүү priority-той task-ууд дотроо Round-Robin-аар ээлжлэх алгоритм.
Quantum	Round-Robin-д task нэг удаа үргэлжлүүлэн ажиллах дээд хугацаа (анхны утга 10 ms).
Ready queue	CPU хүлээж буй, ажиллахад бэлэн task-уудын дараалал.
Response time	Task анх удаа CPU авсан цаг − Arrival.
RR	Round-Robin — task бүр ээлжлэн ≤ quantum хугацаагаар ажиллах алгоритм.
SJF	Shortest Job First — дараагийн CPU burst хамгийн богиныг сонгох алгоритм.
SMP	Symmetric Multiprocessing — олон CPU-тай орчин; task id-г атомаар олгох шалтгаан.
SRTF	Shortest Remaining Time First — SJF-ийн preemptive хувилбар (зөвхөн симуляторт).

Нэр томъёо	**Тайлбар**
Starvation	Бага priority-той task байнга хойш тавигдаж хэзээ ч ажиллахгүй байх эрсдэл.
Turnaround time	Completion − Arrival: task ирснээс дуустал өнгөрсөн хугацаа.
Waiting time	Turnaround − Burst: task ready queue-д хүлээсэн хугацаа.
Aging	Task-ийн хүлээсэн хугацаа өсөх тутам priority-г нэмэгдүүлж starvation-аас сэргийлэх арга (энэ ажилд хэрэгжээгүй).
Атомар үйлдэл	Хооронд нь таслагдах боломжгүй, нэг дор гүйцэтгэгдэх үйлдэл (жишээ нь `__sync_fetch_and_add`).
Dispatcher	Scheduler-ийн сонгосон процесст CPU-г бодитоор шилжүүлэх хэсэг.
Nonpreemptive / Preemptive	Хуваалтгүй: task CPU-г өөрөө чөлөөлөх хүртэл барина. Таслалттай: үйлдлийн систем task-ийг CPU-аас түр салгаж болно.
Race condition	Хоёр thread хуваалцсан өгөгдлийг зэрэг өөрчлөхөд үр дүн нь гүйцэтгэлийн дарааллаас хамаарах алдаа.
Scheduler	Ready queue-ээс дараагийн task-ийг сонгох үйлдлийн системийн хэсэг.

Хураангуй

Энэхүү ажлаар «Operating System Concepts» сурах бичгийн 5-р бүлгийн «Scheduling Algorithms» Programming Project-ийг гүйцэтгэв. Төслийн үндсэн зорилго нь үйлдлийн системийн CPU scheduler хэрхэн ажилладгийг зөвхөн уншиж танилцах бус, өөрсдөө програмчилж, ижил өгөгдөл дээр харьцуулан ажиглаж, үр дүнг тооцоолол ба диаграммаар батлах явдал юм.

Ажлын хүрээнд FCFS, SJF, Priority, Round-Robin, Priority with Round-Robin гэсэн таван алгоритмыг C хэл дээр номын санал болгосон бүтцийн дагуу (`driver.c`, `CPU.c`-ийн `run()`, холбоост жагсаалт, алгоритм бүрт тусдаа файл, `Makefile`) хэрэгжүүлсэн. Номын «Further Challenges» хэсгийн хоёр нэмэлт даалгаврыг гүйцэтгэсэн: (1) task id-г `__sync_fetch_and_add`-аар атомаар олгож, шаардлагатай эсэхийг thread-ийн туршилтаар нотолсон; (2) алгоритм бүрийн дундаж waiting, turnaround, response time-ийг тооцдог.

Багшийн оноосон Gantt chart дүрслэлийг хөтөч дээр ажилладаг интерактив симуляторoор хэрэгжүүлсэн. Симулятор нь C хэрэгжилтээс тусдаа логиктой бөгөөд C програмын бодит гаралтыг уншиж Gantt зурах боломжтой тул хоёр хэрэгжилт бие биенийхээ үр дүнг баталгаажуулдаг.

Тайланд алгоритм бүрийг эхлээд онолын түвшинд тайлбарлаж, номын жишээ (T1…T5) дээр гараар бодож, дараа нь програмын гаралттай тулгасан. Үр дүнгийн хураангуй (дундаж waiting time, ms): FCFS 44.00, SJF 30.00, Priority 33.00, Round-Robin 57.00, Priority+RR 35.00. Автомат тестийн 18 шалгалтын 18 нь давсан. Нэмэлтээр quantum-ын утгыг 1-ээс 30 хүртэл өөрчилж, quantum болон дундаж үзүүлэлтийн хамаарлыг судалсан бөгөөд waiting time нь quantum-ыг багасгахад үргэлж муудаад байдаггүй гэдгийг (жишээлбэл $q = 5$ үед 57.00, $q = 8$ үед 63.00) тоогоор харуулсан.

1. Удиртгал

1.1 Суурь ойлголт

Орчин үеийн компьютерт нэгэн зэрэг олон процесс ажиллаж байдаг боловч нэг CPU цөм нэг мөчид зөвхөн нэг процессыг гүйцэтгэнэ. Иймд ажиллахад бэлэн болсон процессуудаас аль нэгийг нь CPU-д олгох шийдвэрийг үйлдлийн систем тасралтгүй гаргах хэрэгтэй. Энэ шийдвэрийг гаргадаг хэсгийг **CPU scheduler** (богино хугацааны scheduler) гэнэ. Сонгогдсон процесст CPU-г бодитоор шилжүүлэх үйлдлийг **dispatcher** гүйцэтгэнэ.

Ажиллахад бэлэн, гэвч CPU хүлээж буй процессуудыг **ready queue**-д хадгална. Ready queue нь заавал FIFO дараалал байх албагүй: алгоритмаас хамаарч эрэмбэлэгдсэн жагсаалт, priority queue эсвэл холбоост жагсаалт байж болно. Энэ ажилд ready queue-г нэг чиглэлтэй холбоост жагсаалтаар (`list.c`) хэрэгжүүлсэн бөгөөд алгоритмуудын гол ялгаа нь «жагсаалтаас аль элементийг сонгох вэ» гэсэн дүрэмд оршино.

Сурах бичгийн 5.1.3-т CPU scheduling-ийн шийдвэр дараах дөрвөн нөхцөлд гардаг гэж үздэг:

(1) Процесс running төлвөөс waiting төлөвт шилжих үед (жишээ нь I/O хүсэлт);

(2) Процесс running төлвөөс ready төлөвт шилжих үед (жишээ нь таслалт, quantum дуусах);

(3) Процесс waiting төлвөөс ready төлөвт шилжих үед (I/O дуусах);

(4) Процесс дуусах үед.

Хэрэв шийдвэр зөвхөн (1) ба (4)-т гарч байвал scheduling-ийг **nonpreemptive** (хуваалтгүй) гэнэ: процесс CPU-г өөрөө чөлөөлөх хүртэл барина. (2) ба (3)-т мөн гарч байвал **preemptive** (таслалттай) гэнэ. Энэ ажлын FCFS, SJF, Priority нь nonpreemptive, харин Round-Robin ба Priority+RR нь quantum дуусахад процессыг таслах учир preemptive шинжтэй.

1.2 Асуудлын томьёолол

Өгөгдсөн: n ширхэг task, task бүр i-д нэр, priority $p_i \in [1, 10]$, CPU burst $b_i \geq 1$ (ms). Бүх task $t = 0$ мөчид нэгэн зэрэг ready queue-д орсон. Нэг CPU, context switch-ийн зардал 0, I/O байхгүй.

Тодорхойлох зүйл: task бүрийг CPU-д ямар дарааллаар, хэдий хугацаагаар олгох вэ (schedule) гэдгийг алгоритм бүрээр гаргаж, үр дүнг дараах үзүүлэлтээр хэмжих: дундаж waiting time, дундаж turnaround time, дундаж response time (3-р бүлгийн 3.1).

1.3 Ажлын зорилго

1. Номын таван алгоритмыг C хэл дээр номын бүтцийн дагуу зөв хэрэгжүүлэх.

2. Номын хоёр нэмэлт даалгаврыг (атомар tid, дундаж үзүүлэлт) гүйцэтгэж, атомар олголт яагаад хэрэгтэйг туршилтаар харуулах.
3. Багшийн оноосон Gantt chart дүрслэлийг интерактив симуляторoор хэрэгжүүлэж, параметр (burst, priority, quantum, arrival) өөрчилж туршиж болдог болгох.
4. Алгоритм бүрийн үр дүнг эхлээд гараар бодож, дараа нь програм ба симуляторын гаралттай тулгаж зөв ажиллагааг баталгаажуулах.
5. Алгоритмуудыг харьцуулж, онолын таамаглал (SJF-ийн оновчтой байдал, Priority-ийн starvation, RR-ийн response time) тоон үр дүнд нийцэж буйг шалгах.

1.4 Хамрах хүрээ ба таамаглал

- Бүх task $t = 0$ үед ирнэ (номын нөхцөл). Иймд C хэрэгжилтэд arrival time байхгүй.
- Priority 1–10, **их тоо = өндөр priority**. Quantum-ын анхны утга 10 ms (номд заасан).
- Task-ууд зөвхөн CPU ашиглана; I/O болон блоклогдох төлөв хэрэгжүүлээгүй.
- Context switch-ийн зардлыг 0 ms гэж авсан; харин context switch-ийн *тоог* тусад нь тоолж харьцуулсан (8-р бүлэг).
- Симулятор нь номын шаардлагаас гадна arrival time болон preemptive хувилбарууд (SRTF, preemptive Priority)-ыг нэмэлтээр дэмждэг; эдгээр нь C хэрэгжилтэд ороогүй.

1.5 Тайлангийн бүтэц

2-р бүлэг номын шаардлагыг задлан нэгтгэнэ. 3-р бүлэг алгоритм бүрийн онол, гараар бодсон жишээг өгнө. 4–5-р бүлэг C хэрэгжилтийн зохиомж ба код тус бүрийн тайлбар, 6-р бүлэг симулятор, 7-р бүлэг туршилтын арга, 8-р бүлэг үр дүн, 9-р бүлэг хэлэлцүүлэг, 10-р бүлэг дүгнэлт юм. Хавсралтад төслийн ерөнхий мэдээлэл, тестийн төлөвлөгөө болон шаардлагын биелэлтийг нэгтгэв.

2. Сурах бичгийн шаардлага

Энэ бүлэгт номын «Scheduling Algorithms» төслийн шаардлагыг (хуудас P-29…P-31) нэг бүрчлэн задалж, шаардлага бүрийг бид хэрхэн ойлгосон, хаана хэрэгжүүлснийг харуулна. Шаардлага бүрийн нотолгоог Хавсралт B-д файл, мөрийн түвшинд нэгтгэсэн.

2.1 Шаардлагын задаргаа

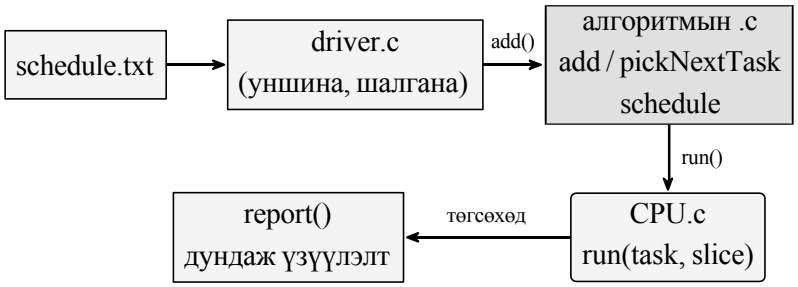
Хүснэгт 2.1. Номын шаардлага ба бидний хэрэгжүүлэлт

№	Номын шаардлага	Бидний ойлголт ба шийдвэр
R1	Таван алгоритм: FCFS, SJF, Priority, RR, Priority with RR	Алгоритм бүрт нэг `.c` файл (`fcfs.c`, `sjf.c`, `priority.c`, `rr.c`, `priority_rr.c`); бүгд ижил интерфэйстэй.
R2	Priority 1–10, *их тоо = өндөр*	`task.h`-д `MIN_PRIORITY=1`, `MAX_PRIORITY=10`; `driver.c` хүрээнээс гадуурхыг татгалзана. Сонгохдоо *хамгийн их* утгатайг авна.
R3	RR-ийн time quantum 10 ms	`QUANTUM=10` анхны утга; нэмэлтээр командын мөрийн аргументаар өөрчилж болно (1–1000).
R4	Оролт `schedule.txt`: `[name], [priority], [burst]`	`driver.c` мөр бүрийг `sscanf`-аар задлан шалгана; `#` тайлбар, хоосон мөрийг алгасна.
R5	Driver нь task-уудыг жагсаалтад оруулж `schedule()`-ийг дуудна	`driver.c`: `add()` → `schedule()` → `report()`.
R6	Сонгосон task-ийг `CPU.c`-ийн `run()` гүйцэтгэнэ; «Running task = [T1] [4] [20] for 20 units.» хэлбэрээр хэвлэнэ	`CPU.c`: `run(task, slice)` нь хэвлэж, цаг ахиулж, `start/finish` тэмдэглэнэ.
R7	Алгоритмыг `Makefile`-аар сонгоно: `make fcfs`, `./fcfs schedule.txt`	`Makefile`-д нэг дүрмээр таван executable үүснэ.
R8	Нэмэлт 1: task id-г SMP орчинд race condition-гүй олгох (atomic integer)	`tasks.c`: `__sync_fetch_and_add`; `tests/tid_race_test.c`-ээр thread-ийн туршилт.
R9	Нэмэлт 2: алгоритм бүрийн дундаж turnaround, waiting, response time	`tasks.c`: `report()`.

2.2 Номын санал болгосон бүтэц

Ном нь хэрэгжилтийг дараах хэсгүүдэд хуваахыг санал болгодог (Зураг 2.1). Алгоритмаас хамаарахгүй хэсэг (`driver.c`, `CPU.c`, жагсаалт) нэг удаа бичигдэж, алгоритм бүр зөвхөн `add()`, `pickNextTask()`, `schedule()` гэсэн гурван функцийг өөрөөр хэрэгжүүлнэ.

Ингэснээр «дүрмийг солих» нь «нэг файлыг солих» болж хувирдаг.



Зураг 2.1. Номын санал болгосон бүтэц. Саарал хайрцаг нь алгоритм бүрт өөр хэрэгжилттэй хэсэг.

2.3 Номд тодорхой бус, бидний шийдсэн асуудлууд

Ном зарим нарийн зүйлийг нээлттэй үлдээдэг. Бид дараах шийдвэрүүдийг гаргаж, тайландаа тодорхой тэмдэглэсэн:

1. **Тэнцүү утгын сонголт (tie-breaking).** SJF, Priority-д хоёр task-ийн burst эсвэл priority тэнцүү бол файлд *эхэнд* орсон task-ийг сонгоно (5.6.1-р хэсэг).
2. **Priority+RR-ийн ээлж.** Тэнцүү priority-той task-уудын ээлжийг жагсаалтын сүүлд дахин оруулах замаар хэрэгжүүлсэн.
3. **SJF-д «burst» гэж юуг ойлгох.** Бүх task $t = 0$ үед ирж, nonpreemptive учир анхны burst ба үлдсэн burst адил; бид анхны `burst`-ыг харьцуулдаг.
4. **Wait/Response-ийн тодорхойлолт.** Arrival $= 0$ тул waiting $=$ finish $-$ burst, response $=$ start (3.1).

3. Онолын үндэс

3.1 Гүйцэтгэлийн үзүүлэлтүүд

Scheduling алгоритмыг дараах гурван хугацаагаар үнэлнэ. Task i-ийн ирсэн цаг a_i, анх CPU авсан цаг s_i, дууссан цаг f_i, burst b_i:

$$\text{Turnaround}_i = f_i - a_i \qquad \text{(ирснээс дуустал нийт хугацаа)}$$
$$\text{Waiting}_i = \text{Turnaround}_i - b_i \qquad \text{(ready queue-д хүлээсэн нийт хугацаа)}$$
$$\text{Response}_i = s_i - a_i \qquad \text{(ирснээс анх CPU авах хүртэл)}$$

Бидний тохиолдолд $a_i = 0$ тул $\text{Turnaround}_i = f_i$, $\text{Response}_i = s_i$. Waiting-ийн томьёо $f_i - b_i$ нь «task-ийн амьдралын турш CPU дээр байсан хугацаа b_i-г хасахад үлдсэн хугацаа бол ready queue-д хүлээсэн хугацаа» гэсэн санаанаас гарна.

Жишээ (FCFS, T3). T3 нь 45-р ms-д эхэлж (response $= 45$), 70-р ms-д дуусна (turnaround $= 70$). Burst 25 тул waiting $= 70 - 25 = 45$. T3 эхлэхээс өмнө хүлээсэн 45 ms бол T1 (20) ба T2 (25)-ийн ажилласан хугацаа тул үр дүн ойлгомжтой.

Яагаад гурвуулаа хэрэгтэй вэ? Turnaround нь «ажил хэр хурдан дууссан»-ыг, waiting нь «scheduling-ээс болж хэр их хугацаа дэмий өнгөрсөн»-ийг, response нь «хэрэглэгч эхний хариуг хэр хурдан харсан»-ыг хэмждэг. Интерактив системд response нь, багцын системд turnaround нь илүү чухал. Алгоритмууд эдгээрийн аль нэгийг нь сайжруулахдаа нөгөөг нь муутгадаг тул гурвыг хамт харна (9-р бүлэг).

3.2 Туршилтын өгөгдөл

Тайланд бүх гараар бодсон жишээг номын өгөгдөл дээр хийнэ (Хүснэгт 3.1). Нийт burst =

Хүснэгт 3.1. Номын жишээ өгөгдөл (`tests/schedule.txt`)

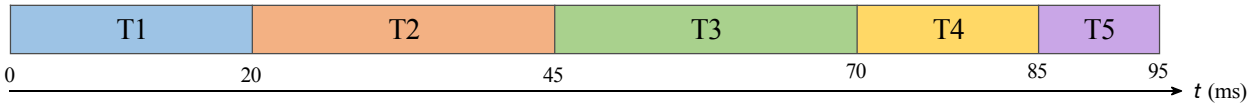
Task	Priority	Burst (ms)
T1	4	20
T2	2	25
T3	3	25
T4	3	15
T5	10	10

$20 + 25 + 25 + 15 + 10 = 95$ ms тул бүх алгоритмын Gantt chart 95-р ms-д дуусна (CPU сул зогсолтгүй).

3.3 FCFS (First-Come, First-Served)

Дүрэм: ready queue-д орсон дарааллаар нь гүйцэтгэнэ; task дуустал таслахгүй. Хэрэгжүүлэхэд хамгийн энгийн (FIFO дараалал).

Гараар бодолт. Дараалал T1, T2, T3, T4, T5. Эхлэх цаг нь өмнөх task-уудын burst-ийн нийлбэр:



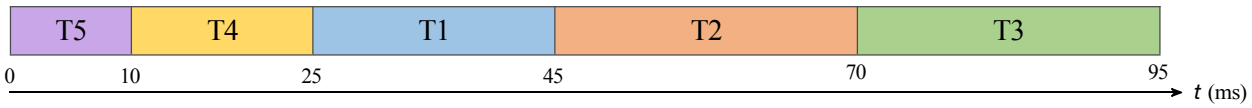
Waiting = 0, 20, 45, 70, 85; дундаж $= \frac{0 + 20 + 45 + 70 + 85}{5} = \frac{220}{5} = 44$.

Сул тал — convoy effect. Хамгийн богино task (T5, 10 ms) хамгийн сүүлд орсон тул 85 ms хүлээв. Урт task урд байх тусам богино task-ууд «цуваанд» гацна. Дундаж waiting нь task-уудын дарааллаас шууд хамаарна.

3.4 SJF (Shortest-Job-First)

Дүрэм: ready queue дахь task-уудаас дараагийн CPU burst хамгийн богиныг сонгоно. Burst тэнцүү бол FCFS-ээр шийднэ (бидний tie-breaking).

Гараар бодолт. Burst-оор эрэмбэлбэл: T5(10), T4(15), T1(20), T2(25), T3(25). T2 ба T3 тэнцүү тул файлд эхэнд байсан T2 түрүүлнэ.



Waiting = 0, 10, 25, 45, 70 (T5, T4, T1, T2, T3); дундаж 150/5=30.

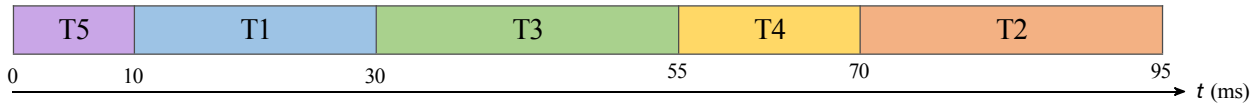
SJF яагаад оновчтой вэ? (солилцооны аргумент). Хэрэв дараалалд урт task i (burst b_i) богино task j (burst $b_j < b_i$)-ийн өмнө, шууд зэрэгцээ байвал тэдгээрийг солилцъё. Дараалалд i нь j-ийн өмнө байхад j нь b_i-ээр илүү хүлээж байв; сольсны дараа j нь b_i-ээр багаар хүлээж, харин i нь b_j-ээр илүү хүлээнэ. Нийт waiting-ийн өөрчлөлт $= b_j - b_i < 0$. Өөрөөр хэлбэл
«урт нь богинынхоо өмнө» гэсэн адил хос бүрийг засах нь нийт waiting-ийг бууруулна. Хос үлдэхгүй болтол (burst өсөх дараалалтай болтол) үүнийг давтвал нийт waiting хамгийн бага болно. Иймд бүх task нэгэн зэрэг ирэх үед SJF дундаж waiting-ийг хамгийн бага болгодог (бидний тооцоо 30-аас бага зүйл FCFS/Priority/RR-д олдохгүй).

Сул тал. Практикт дараагийн CPU burst-ийг *урьдчилан мэдэх* боломжгүй (зөвхөн таамаглана). Мөн урт task-ууд байнга хойш тавигдаж болно.

3.5 Priority scheduling

Дүрэм: task бүрд priority өгөгдсөн; хамгийн өндөр priority-той (бидний тохиолдолд хамгийн *их* тоотой) task-ийг сонгоно. Тэнцүү бол FCFS. SJF бол burst-ийн урвуу хэмжээг priority болгосон тусгай тохиолдол.

Гараар бодолт. Priority буурах дараалал: T5(10), T1(4), T3(3), T4(3), T2(2). T3 ба T4 тэнцүү (3) тул файлд эхэнд байсан T3 түрүүлнэ.



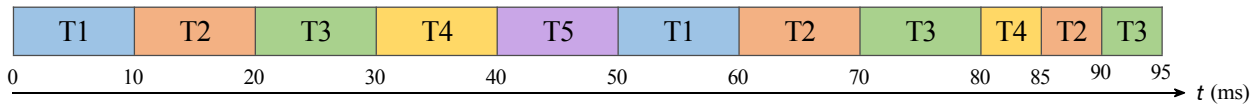
Waiting = $0, 10, 30, 55, 70$; дундаж = $\frac{165}{5} = 33$.

Сул тал — starvation (өлсгөлөн). Бага priority-той task, өндөр priority-той task-ууд байнга ирсээр байвал хэзээ ч CPU авч чадахгүй. Манай жишээнд T2 (priority 2) хамгийн сүүлд ажиллаж 70 ms хүлээв. Шийдэл болох *aging* (хүлээсэн хугацаагаар priority-г аажмаар нэмэх) энэ ажилд хэрэгжээгүй.

3.6 Round-Robin (RR)

Дүрэм: task бүр ээлжлэн дээд тал нь *quantum* q хугацаагаар ажиллана. Quantum дуусахад task дуусаагүй бол ready queue-ийн *сүүлд* орно. Хэрэв task q-ээс богино хугацаа үлдсэн бол үлдсэнээ л ажиллаад дуусна.

Гараар бодолт ($q = 10$). Эхний тойрог: T1, T2, T3, T4, T5 тус бүр 10 ms. T5 (burst 10) дуусна. Хоёр дахь тойрог: T1 (үлдсэн 10) дуусна; T2, T3 нэмж 10; T4 (үлдсэн 5) дуусна. Эцэст нь T2, T3 тус бүр 5 ms.



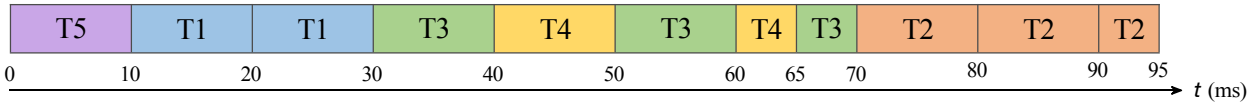
Дууссан цаг: T1 60, T2 90, T3 95, T4 85, T5 50. Waiting = $f - b$: $40, 65, 70, 70, 40$; дундаж = $\frac{285}{5} = 57$. Response = s: $0, 10, 20, 30, 40$; дундаж = 20.

Quantum-ын нөлөө. n task, quantum q үед task бүр дараагийн ээлжээ дээд тал нь $(n-1)q$ хүлээнэ. Хэрэв бүх burst $\geq q$ бол k-р task-ийн response $= (k-1)q$, дундаж response $= \frac{(n-1)q}{2}$. Бидний өгөгдөлд $n = 5, q = 10$: $\frac{4 \cdot 10}{2} = 20$ — гараар бодсон 20-тай таарна. $q = 4$ үед $\frac{4 \cdot 4}{2} = 8$ — C хэрэгжилтийн гаралттай (8.00) таарна. Quantum хэт том ($q \geq$ хамгийн урт burst) болбол RR нь FCFS болж хувирна ($q \geq 25$ үед 44.00, 63.00, 44.00 — 8-р бүлэг). Quantum хэт жижиг бол context switch-ийн тоо огцом өснө: нийт slice-ийн тоо $\sum_i \lceil b_i/q \rceil$, context switch $=$ slice -1 ($q = 10$ үед $2+3+3+2+1-1 = 10$; $q = 4$ үед $26 - 1 = 25$; $q = 1$ үед 94).

3.7 Priority with Round-Robin

Дүрэм: priority-ийн дарааллаар сонгож, *тэнцүү priority-той* task-ууд дотроо RR-аар ээлжилнэ. Өндөр priority-той task байгаа цагт бага priority-той task ажиллахгүй.

Гараар бодолт ($q = 10$). Эхлээд T5 (10) 10 ms-д дуусна. Дараа нь T1 (priority 4, ганцаараа): 10 ms $\times 2$. Дараа нь T3, T4 (priority 3) ээлжилнэ: T3 10, T4 10, T3 10, T4 5 (дуусна), T3 5 (дуусна). Эцэст нь T2 (priority 2) ганцаараа.



Дууссан цаг: T5 10, T1 30, T4 65, T3 70, T2 95. Waiting: 0, 10, 50, 45, 70; дундаж $= \frac{175}{5} = 35$.

Priority-тэй харьцуулбал дундаж waiting (35) нь 33-аас 2 ms-ээр их боловч дундаж response $33 \rightarrow 30$ болж сайжирсан: T4 нь Priority-д 55-р ms-д эхэлдэг байсан бол энд T3-тай ээлжилж 40-р ms-д эхэлнэ. Харин T3-ийн дуусах цаг 55-аас 70 болж хойшилсон нь ээлжлэлтийн «үнэ» юм.

3.8 Алгоритмуудын онолын хураангуй

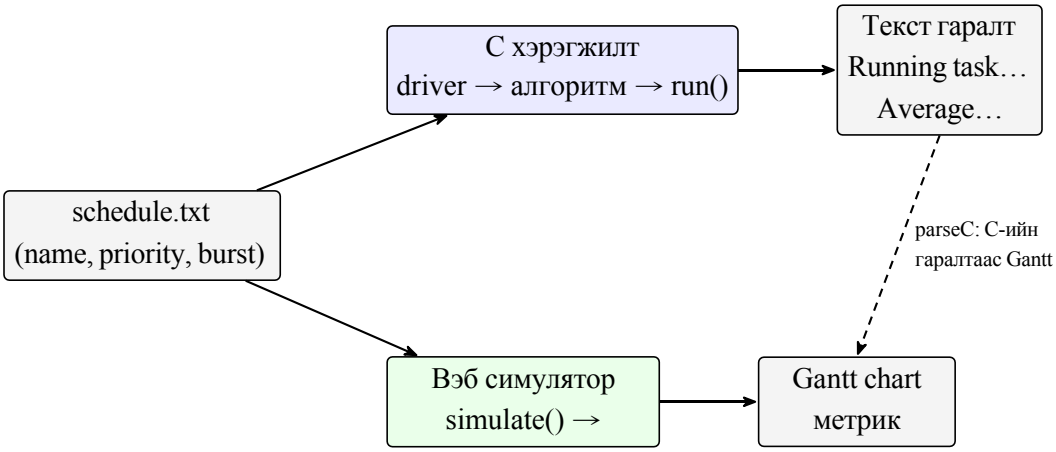
Хүснэгт 3.2. Алгоритмуудын шинж чанарын харьцуулалт

Алгоритм	**Таслалт**	**Starvation**	**Давуу тал**	**Сул тал**
FCFS	үгүй	үгүй	Энгийн, шударга дараалал	Convoy effect, дундаж waiting өндөр
SJF	үгүй	боломжтой	Дундаж waiting оновчтой	Burst-ийг урьдчилан мэдэх хэрэгтэй
Priority	үгүй	**боломжтой**	Чухал task түрүүлнэ	Бага priority өлсөнө
RR	тийм	үгүй	Response сайн, шударга	Context switch их, turnaround өндөр
Priority+RR	тийм	боломжтой	Priority + шударга ээлж	Priority хоорондын starvation үлдэнэ

4. Системийн зохиомж

4.1 Ерөнхий архитектур

Төсөл хоёр бие даасан хэсгээс бүрдэнэ (Зураг 4.1): (1) номын шаардлагыг хангасан C хэрэгжилт; (2) багшийн оноосон Gantt chart дүрслэлтэй вэб симулятор. Хоёр хэсэг нь *ижил оролтын формат* (`schedule.txt`) ашигладаг, мөн симулятор нь C-ийн хэвлэсэн гаралтыг уншиж (`parseC`) Gantt зурах чадвартай. Энэ холбоос нь хоёр хэрэгжилтийг бие биесээр шалгах боломжийг өгдөг.



Зураг 4.1. Системийн ерөнхий архитектур. Тасархай сум нь C-ийн гаралтыг симуляторт уншуулах холбоо.

4.2 C хэрэгжилтийн модулиуд

C хэрэгжилт нь номын загварыг дагана. Алгоритмаас хамаарахгүй дөрвөн файл (`driver.c`, `CPU.c`, `list.c`, `tasks.c`) нэг удаа бичигдэж, алгоритм тус бүрийн нэг файл (`fcfs.c`, `sjf.c`, `priority.c`, `rr.c`, `priority_rr.c`) `schedulers.h`-д зарласан ижил гурван функцийг хэрэгжүүлнэ.

4.3 Өгөгдлийн бүтэц

4.3.1 Task бүтэц

Task бүрийг `Task` бүтцээр илэрхийлнэ (Хүснэгт 4.2). Номын заасан `name`, `tid`, `priority`, `burst` талбаруудаас гадна `remaining`, `start`, `finish` талбарыг нэмсэн нь үзүүлэлтүүдийг `run()` дотор шууд тооцоход шаардлагатай.

Хүснэгт 4.1. Хэрэгжилтийн файлууд ба тэдгээрийн үүрэг

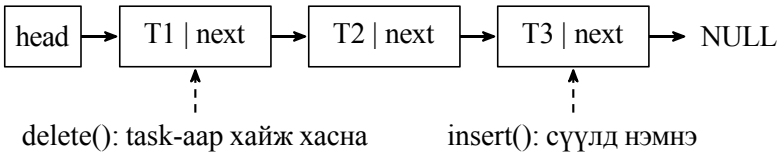
Файл	**Үүрэг**
`task.h`	`Task` бүтэц, тогтмолууд (`MIN_PRIORITY`, `MAX_PRIORITY`, `QUANTUM`, `MAX_TASKS`), `new_task()`, `report()`-ийн зарлал
`schedulers.h`	Алгоритм бүрийн хэрэгжүүлэх интерфэйс: `add()`, `schedule()`, `pickNextTask()`, `quantum`
`driver.c`	`schedule.txt` уншиж шалгах (priority 1–10, burst ≥ 1, формат), quantum аргумент; `add()` → `schedule()` → `report()`
`list.c / list.h`	Холбоост жагсаалт: `insert` (сүүлд нэмнэ), `delete` (тухайн task-ийг хасна)
`CPU.c / CPU.h`	`run(task, slice)`: «Running task = …» хэвлэж, simulated цаг ахиулж, `start`/`finish` тэмдэглэнэ
`tasks.c`	`new_task()`: атомар tid, бүх task-ийн бүртгэл; `report()`: waiting/turnaround/response ба дундаж
`fcfs.c` … `priority_rr.c`	`pickNextTask()` болон `schedule()`-ийн алгоритм тус бүрийн хэрэгжилт
`Makefile`	Алгоритм бүрийг нийтлэг хэсэгтэй холбож тусдаа executable үүсгэх; `make test`
`src/web/index.html`	Симулятор (нэг файл): оролтын хүснэгт → `simulate()` → Gantt, метрик, алхамчилсан тоглуулалт

Хүснэгт 4.2. `Task` бүтцийн талбарууд

Талбар	**Төрөл**	**Утга**
`name`	`char *`	Task-ийн нэр (heap дээр `strdup`-аар хуулсан)
`tid`	`int`	Давтагдашгүй id, атомаар олгогдоно
`priority`	`int`	1–10, их тоо = өндөр
`burst`	`int`	Анхны CPU burst (ms); үзүүлэлт тооцоход хэрэглэнэ
`remaining`	`int`	Үлдсэн burst; RR-д дахин дахин буурна, 0 болоход дуусна
`start`	`int`	Анх CPU авсан цаг; −1 бол хараахан ажиллаагүй (response тооцоход)
`finish`	`int`	Дууссан цаг (turnaround, waiting тооцоход)

4.3.2 Холбоост жагсаалт

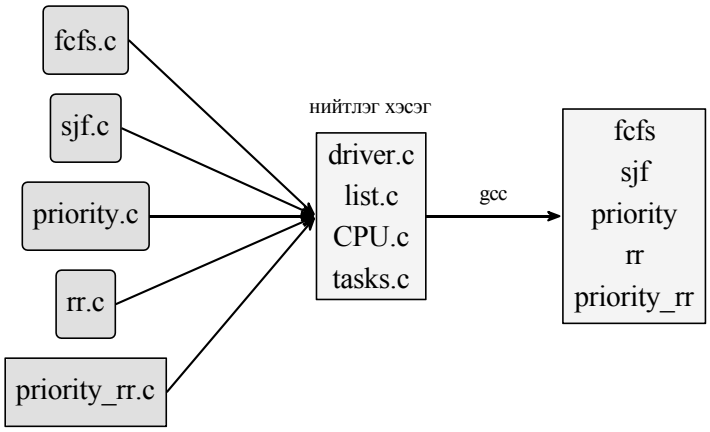
Ready queue-г нэг чиглэлтэй холбоост жагсаалтаар хэрэгжүүлсэн (Зураг 4.2). `insert` нь жагсаалтын *сүүлд* нэмдэг тул жагсаалтын дараалал нь файлд орсон дараалал (FCFS-ийн дараалал) болно. Энэ шинж чанар нь tie-breaking-ийг «жагсаалтад эхэнд байсан» гэж энгийнээр хэрэгжүүлэх боломжийг олгодог.



Зураг 4.2. Ready queue — холбоост жагсаалт. `insert` нь сүүлд, `delete` нь тухайн task-ийг хайж хасна.

4.4 Нэг интерфэйс — таван хэрэгжилт

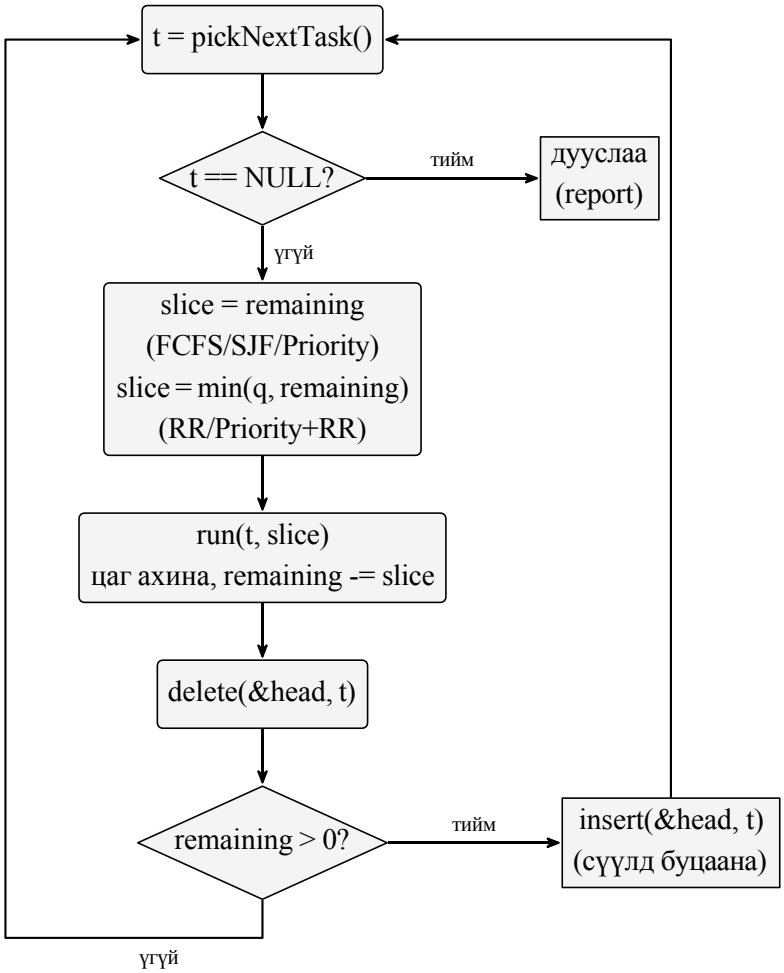
Алгоритм бүр `schedulers.h`-д зарласан гурван функцийг өөрийнхөөрөө хэрэгжүүлнэ. `driver.c` нь ямар алгоритмтай холбогдсоноо мэддэггүй, зөвхөн `add()`-аар task оруулж `schedule()`-ийг дууддаг. Алгоритмыг *хөрвүүлэх үед* `Makefile` сонгоно (Зураг 4.3), иймээс програм ажиллах үед «аль алгоритм вэ» гэж шалгах нөхцөл (`if`/`switch`) огт байхгүй.



Зураг 4.3. `Makefile`: алгоритмын нэг файл + нийтлэг хэсэг = нэг executable. Таван удаа хөрвүүлж таван програм гарна.

4.5 Гүйцэтгэлийн урсгал

Бүх алгоритмын `schedule()` дотор «сонго → ажиллуул → жагсаалтаас хас (хэрэв үлдсэн бол сүүлд нь буцаа)» гэсэн нэг ижил гогцоо байна (Зураг 4.4). Алгоритмуудын *цорын ганц* жинхэнэ ялгаа нь (a) `pickNextTask()`-ийн сонголтын дүрэм, (б) slice-ийн урт (remaining эсвэл min(*q*, remaining)) юм.



Зураг 4.4. `schedule()`-ийн гогцоо. Nonpreemptive алгоритмуудад «remaining > 0» нь хэзээ ч үнэн болдоггүй (task нэг дор дуусна).

4.6 Симуляторын зохиомж

Симулятор нь `src/web/index.html` нэг файл (HTML + CSS + JavaScript), гадны сангүй. Зохиомжийн гол санаа: *нэг цагийн алхам = 1 ms*. `simulate()` нь цаг тутамд «хэн ажиллаж байна, хэн ready queue-д байна, юу болсон (PICK/RUN/PREEMPT/DONE)» гэдгийг `log[t]` массивт хадгалдаг. Gantt chart, метрик, ready queue, pseudo-code-ийн тодруулалт бүгд энэ

`log`-оос *зөвхөн унших* замаар зурагддаг тул slider-аар цагийг урагш хойш нь чөлөөтэй хөдөлгөх боломжтой. Дэлгэрэнгүйг 6-р бүлэгт үзнэ.

5. Хэрэгжилт

Энэ бүлэгт C кодын гол хэсгүүдийг *яагаад ингэж бичсэн* гэдэг шалтгаантай нь хамт тайлбарлана. Бүрэн код `src/c/` хавтсанд байна.

5.1 Хэл, орчин

C (gnu11 стандарт, `gcc -Wall -Wextra -O2`), GNU Make. Симулятор нь гадны сангүй цэвэр HTML/CSS/JavaScript нэг файл. Хөрвүүлэх үед анхааруулга гарахгүй байх шаардлага тавьсан (`-Wall -Wextra`).

5.2 task.h ба tasks.c: Task ба тооцоолол

5.2.1 Task үүсгэх ба атомар tid

```
static Task *all_tasks[MAX_TASKS];
static int task_count = 0;
static int next_tid = 1;

Task *new_task(char *name, int priority, int burst)
    { Task *t = malloc(sizeof *t);
    t->name = strdup(name);
    t->tid = __sync_fetch_and_add(&next_tid, 1);
    t->priority = priority;
    t->burst = t->remaining = burst;
    t->start = -1;
    t->finish = 0;
    all_tasks[task_count++] = t;
    return t;
}
```

- `strdup(name)`: driver-ийн `name` буфер дараагийн мөрийг уншихад дарагдах тул task өөрийн хуулбартай байх ёстой.
- `start = -1`: «хараахан ажиллаагүй» гэсэн тэмдэг. `run()` нь анхны дуудлагад л `start`-ыг онооно. 0-ийг ашиглаж болохгүй, учир нь $t = 0$-д эхэлсэн task-ийн жинхэнэ response нь 0 байдаг.
- `all_tasks[]`: task-уудыг *үүссэн дарааллаар* бүртгэнэ. Алгоритмууд task-ийг жагсаалтаас хасдаг учир эцсийн `report()`-д бүх task-ийг файлын дарааллаар хэвлэхийн тулд тусдаа бүртгэл хэрэгтэй.

5.2.2 Яагаад атомар? (Нэмэлт даалгавар 1)

SMP орчинд CPU бүр өөрийн scheduler-ийг ажиллуулж болно. Хоёр CPU нэгэн зэрэг `new_task()` дуудвал `tid = next_tid++` нь «уншина → нэмнэ → бичнэ» гэсэн гурван алхамтай тул хоёулаа ижил утгыг уншиж *ижил tid* авч болно (race condition). `__sync_fetch_and_add(&next_tid, 1)` нь GCC-ийн builtin бөгөөд эдгээр гурван алхмыг хоорондоо таслагдахгүй нэг үйлдлээр гүйцэтгэж, нэмэхээс *өмнөх* утгыг буцаана. Иймээс tid 1-ээс эхэлнэ, давхардахгүй.

Туршилт. Манай драйвер нэг thread-тэй тул tid-ийн давхардал бодитоор гарч харагдахгүй. Иймд `tests/tid_race_test.c` туршилтыг бичсэн: 8 thread тус бүр нэг тоолуурыг 200 000 удаа нэмэгдүүлнэ. Нэг тоолуур энгийн `x = x + 1`, нөгөөг нь `__sync_fetch_and_add`-аар нэмнэ. Хүлээгдэх эцсийн утга $8 \times 200000 + 1 = 1600001$. Үр дүнг 8.7-р хэсэгт харуулсан.

Хязгаарлалт (чухал). Атомар болгосон зүйл зөвхөн `next_tid`. Харин task-уудын бүртгэл (`all_tasks[]`) бодит SMP орчинд lock-гүй race-тэй. Манай драйвер нэг thread-тэй тул энэ нь аюулгүй (кодод тэмдэглэгдсэн), гэвч олон thread-ээр `new_task()` дуудах бол энэ бүртгэлийг mutex-ээр хамгаалах шаардлагатай. Ном зөвхөн tid-ийн race-ийг шаарддаг тул үүнийг бид хэрэгжүүлсэн.

5.2.3 report(): дундаж үзүүлэлтүүд (Нэмэлт даалгавар 2)

```
int tat = t->finish, wait = tat - t->burst, resp = t->start;    /* arrival = 0
    */
...
tw += wait; tt += tat; tr += resp;
...
printf("Average waiting time     = %.2f\n", tw / task_count);
```

Томьёонууд нь 3.1-д гаргасантай яг адил ($a_i = 0$). Нийлбэрийг `double`-д хадгалах нь бүхэл тоон хуваалтаар бутархай хэсгийг алдахаас сэргийлнэ (жишээ нь 60.40).

5.3 list.c: холбоост жагсаалт

```
void insert(struct node **head, Task *task) {
    struct node *n = malloc(sizeof *n), *p;
    n->task = task; n->next = NULL;
    if (!*head) { *head = n; return; }
    for (p = *head; p->next; p = p->next) ;
    p->next = n;
}
```

```
void delete(struct node **head, Task *task) {
    struct node *p = *head, *prev = NULL;
    while (p && p->task != task) { prev = p; p = p->next; }
    if (!p) return;
    if (prev) prev->next = p->next; else *head = p->next;
    free(p);
}
```

- `struct node **head` (заагчийн заагч): жагсаалтын эхний элементийг өөрчлөх (хоосон жагсаалтад нэмэх, эхнийхийг хасах) үед дуудагчийн `head`-ийг шууд шинэчилнэ.
- `delete` нь элементийг *task-ийн заагчаар* хайдаг (нэрээр биш). Хоёр task ижил нэртэй байж болох тул заагчаар таних нь зөв.
- `delete` зөвхөн *node*-ийг `free` хийдэг, `Task`-ийг биш: task нь `all_tasks[]`-д үлдэж, эцсийн `report()`-д хэрэгтэй.
- Сул тал: `insert` нь $O(n)$ (сүүл хүртэл алхана). $n \leq 1000$ тул асуудалгүй; tail заагч нэмбэл $O(1)$ болно.

5.4 CPU.c: run()

```
static int now = 0;   /* simulated clock (ms) */

void run(Task *task, int slice) {
    printf("Running task = [%s] [%d] [%d] for %d units.  (t = %d..%d)\n",
           task->name, task->priority, task->remaining, slice, now, now +
    slice);
    if (task->start < 0) task->start = now;
    now += slice;
    task->remaining -= slice;
    if (task->remaining == 0) task->finish = now;
}
```

`run()` нь жинхэнэ CPU-г загварчилна: бодит хугацаа хүлээхгүй, зөвхөн `now` хувьсагчийг `slice`-аар ахиулна. Үзүүлэлт тооцоход шаардлагатай бүх мэдээлэл (`start`, `finish`) энд л бүртгэгдэнэ. Номын заасан хэвлэлтийн формат («Running task = [T1] [4] [20] for 20 units.») хадгалагдсан бөгөөд бид нэмэлтээр `(t = a..b)` хугацааг хавсаргасан нь симулятор С-ийн гаралтыг уншихад шаардлагатай.

5.5 driver.c: оролтыг шалгах

Driver нь `schedule.txt`-ийн мөр бүрийг `sscanf`-аар задална:

```
if (sscanf(s, " %63[^,], %d , %d %c", name, &pr, &bu, &extra) != 3) { /*
    алдаа */ }
```

Формат `" %63[^,], %d , %d %c"`: нэрийг таслал хүртэл (63 тэмдэгтээс ихгүй) уншина; дараа нь хоёр бүхэл тоо; хамгийн сүүлд `%c` нь *илүү тэмдэгт* байгаа эсэхийг шалгана. Зөв мөрөнд 3 утга л задарна (`= 3`); мөрийн төгсгөлд илүү тэмдэгт байвал 4 утга задарч мөрийг татгалзана. Ингэснээр «T1, 4, 20 xyz» мэт мөр зөвшөөрөгдөхгүй.

Хүснэгт 5.1. Оролтын шалгалт ба алдааны хариу

Шалгах зүйл	**Алдааны мэдээлэл**	**Тест файл**
Мөрийн формат буруу	`expected "name, priority, burst" but found: …`	`bad_format.txt`
Priority 1–10-аас гадуур	`priority 11 is outside 1..10.`	`bad_priority.tx`
Burst < 1	`CPU burst must be at least 1`	`bad_burst.txt`
Task огт байхгүй	`contains no tasks.`	`bad_empty.txt`
Quantum бүхэл биш, <1 эсвэл >1000	`quantum must be an integer from 1 to 1000`	командын мөр
Файл байхгүй	`perror`-ийн мэдээлэл	`missing.txt`
`MAX_TASKS` (1000)-аас их task	`more than 1000 tasks.`	—

Алдаа бүрт мөрийн дугаар болон шалтгааныг хэлж, `stderr`-д бичээд *тэг биш* exit code-оор гарна. Ингэснээр скрипт (`run_tests.sh`) алдааг автоматаар илрүүлж чадна.

5.6 Алгоритмын файлууд

5.6.1 Tie-breaking: яагаад «<» гэж хатуу харьцуулсан бэ?

Priority болон SJF-ийн `pickNextTask()` нь жагсаалтыг эхнээс нь дуустал алхаж «одоогоор хамгийн сайн»-ыг санана:

```
/* sjf.c */
Task *best = NULL;
for (struct node *p = head; p; p = p->next)
    if (!best || p->task->burst < best->burst) best = p->task;
```

```
return best;

/* priority.c: ижил, харьцуулалт нь  >  */
if (!best || p->task->priority > best->priority) best = p->task;
```

Харьцуулалт *хатуу* (< эсвэл >, ≤ биш) тул тэнцүү утгатай хойно орших task нь «хамгийн сайн»-ыг сольж чадахгүй. Тиймээс тэнцүү үед *жагсаалтад эхэнд* байсан task үлдэнэ. Хэрэв ≤ гэж бичсэн бол сүүлд орсон нь сонгогдох байсан. Манай жишээнд SJF-ийн T2 ба T3 (burst 25), Priority-ийн T3 ба T4 (priority 3) нь яг ийм тохиолдол бөгөөд 3-р бүлгийн гараар бодолтыг энэ дүрмээр хийсэн.

5.6.2 FCFS

```
Task *pickNextTask(void) { return head ? head->task : NULL; }
void schedule(void) {
    Task *t;
    while ((t = pickNextTask())) { run(t, t->remaining); delete(&head, t); }
}
```

Жагсаалтын *эхний* элементийг сонгож, `remaining`-ийг бүтнээр нь ажиллуулна (slice = remaining ⇒ таслалтгүй). Task дуусаад жагсаалтаас хасагдана.

5.6.3 Round-Robin

```
void schedule(void)
    { Task *t;
    while ((t = pickNextTask())) {
        int slice = t->remaining < quantum ? t->remaining : quantum;
        run(t, slice);
        delete(&head, t);
        if (t->remaining > 0) insert(&head, t);
    }
}
```

- `slice = min(quantum, remaining)`: quantum-аас богино үлдэгдэлтэй task үлдсэнээ л ажиллаад дуусна, CPU-г дэмий барихгүй.
- `delete` дараа нь `insert`: дуусаагүй task-ийг жагсаалтын *сүүлд* шилжүүлэх нь «дараагийн ээлжиндээ хүлээнэ» гэсэн утгатай. Жагсаалтыг эргүүлэх (rotate) тусгай функц бичихгүйгээр байгаа хоёр үйлдлийг дахин ашигласан.

5.6.4 Priority with Round-Robin

`priority_rr.c`-ийн `pickNextTask()` нь `priority.c`-тэй ижил, `schedule()` нь `rr.c`-тэй ижил. Хоёрыг хослуулахад нэмэлт код хэрэггүй болсон шалтгаан: дуусаагүй task сүүлд орсноор *тэнцүү priority-той task-уудын дунд* хамгийн сүүлд ажилласан нь жагсаалтын хамгийн сүүлд байх тул «эхэнд байсан тэнцүү priority-тойг сонгох» дүрэм өөрөө ээлжийг бий болгодог. Харин өндөр priority-той task байгаа үед бага priority-той task-ууд жагсаалтад үлдэж дараалж хүлээнэ.

Шалгалт (`schedule_ties.txt`, $q = 4$). A, B, C (priority 5; burst 12, 7, 3) ба D (priority 1; burst 9). Хүлээгдэх дараалал: A4, B4, C3 (C дууслаа), A4, B3 (B дууслаа), A4 (A дууслаа), дараа нь D 4, 4, 1. Програмын гаралт ($t = 0..31$): `A 0..4, B 4..8, C 8..11, A 11..15, B 15..18, A 18..22, D 22..26, 26..30, 30..31` — яг таарсан. Дундаж waiting $= \dfrac{10 + 11 + 8 + 22}{4} = 12.75$.

5.7 Makefile

```
ALGS    = fcfs sjf priority rr priority_rr
COMMON  = driver.c list.c CPU.c tasks.c
$(ALGS): %: %.c $(COMMON) $(HDRS)
        $(CC) $(CFLAGS) -o $@ $< $(COMMON)
```

Нэг *static pattern rule* таван алгоритмыг бүгдийг хөрвүүлнэ: `$@` нь executable-ийн нэр (жишээ нь `rr`), `$<` нь түүнд харгалзах `rr.c`. Header файлууд хамаарлын жагсаалтад байгаа тул `.h` өөрчлөгдөхөд дахин хөрвүүлнэ. `make test` нь бүгдийг хөрвүүлж `run_tests.sh`-ийг ажиллуулна.

5.8 Симуляторын хэрэгжилт

Симуляторын цөм, UI-ийн хэрэгжилтийг 6-р бүлэгт тусад нь тайлбарлав.

6. Gantt chart симулятор

Багшийн оноосон UI дүрслэл нь Gantt chart (waiting, turnaround, response time-тай). Энэ бүлэгт симуляторын ажиллагаа, хэрэгжилтийн санаа, хэрэглэгчийн боломжуудыг тайлбарлана. Хэрэглэгчийн дэлгэрэнгүй заавар тусдаа Manual баримтад бий.

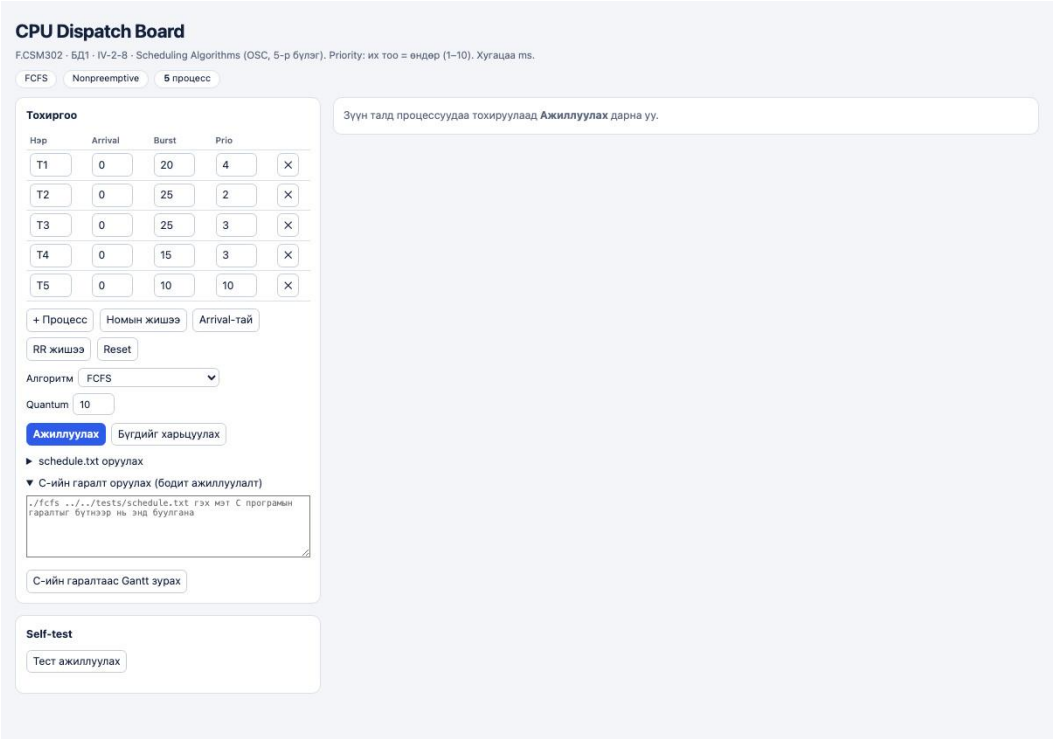
6.1 Симуляторын боломжууд

- **Процессын хүснэгт:** нэр, arrival, burst, priority-г засах, мөр нэмэх, устгах. «schedule.txt оруулах» хэсэгт номын форматаар (`name, priority, burst`) буулгаж болно.
- **Алгоритм ба quantum:** 7 алгоритм — таван үндсэн алгоритм дээр нэмэлтээр SRTF (preemptive SJF) ба preemptive Priority. Quantum нь RR, Priority+RR-д л идэвхтэй.
- **Gantt chart:** дээд мөр CPU (процесс бүр өөр өнгөтэй, idle нь судалтай); процесс бүр тусдаа мөртэй — тод хэсэг нь CPU дээр ажилласан, бүдэг шугам нь ready queue-д хүлээсэн үе (waiting). Улаан босоо шугам нь одоогийн хугацаа.
- **Алхамчилсан тоглуулалт:** slider, өмнөх/дараагийн алхам, эхлэл/төгсгөл рүү үсрэх товч, «Тоглуулах». Алхам бүрд pseudo-code-ийн биелж буй мөр тодорч, PICK / RUN / PREEMPT / DONE тайлбар, context switch, дууссан тоо, ready queue харагдана.
- **Метрикийн хүснэгт** ба **«Бүгдийг харьцуулах»**: 7 алгоритмын дунджийг зэрэгцүүлнэ.
- **C-ийн гаралт оруулах:** C програмын гаралтыг буулгахад UI тэр *бодит гаралтаас* Gantt зурна.
- **Self-test:** симулятор доторх 12 автомат шалгалт.

6.2 simulate(): 1 ms-ийн алхамтай цөм

Симуляторын цөм `simulate(P, alg, q)` нь цагийг 0-ээс эхлэн 1 ms-ээр ахиулдаг. Алхам бүрд дараах дарааллаар шийдвэр гаргана:

1. **Ирэлт:** t мөчид ирсэн (arrival $\leq t$) процессуудыг ready queue-ийн сүүлд нэмнэ.
2. **Таслалт шалгах:** одоо ажиллаж буй процесс байвал алгоритмын таслалтын нөхцөл биелсэн эсэхийг шалгана:
    - **SRTF:** ready queue-д үлдсэн хугацаа нь одоогийнхоос бага процесс байвал;
    - **preemptive Priority:** илүү өндөр priority-той процесс байвал;
    - **RR:** quantum дууссан бол;
    - **Priority+RR:** quantum дууссан *эсвэл* илүү өндөр priority-той процесс ирсэн бол.



Зураг 6.1. Симулятор — анхны төлөв. Зүүн талд процессын хүснэгт, алгоритмын сонголт, баруун талд (ажиллуулсны дараа) Gantt chart, метрик гарна.

Таслагдсан процесс ready queue-ийн *сүүлд* орно.

3. **Сонголт:** CPU сул бол алгоритмын дүрмээр ready queue-ээс сонгоно (`best()`: хамгийн бага түлхүүртэйг, тэнцүү бол жагсаалтад эхэнд байсныг).
4. **Гүйцэтгэл:** сонгогдсон процессын `rem`-ийг 1-ээр бууруулна; 0 болбол дууссан гэж тэмдэглэж `comp[i]=t+1` хадгална.
5. **Лог:** цаг тутмын төлөв (`run`, `pr` — таслагдсан, `pk` — сонгогдсон, `rem`, `queue`, `fin`) `log`-д нэмэгдэнэ.

Цикл бүх процесс дуустал давтагдана. Эцэст нь `rows` дахь waiting/turnaround/response-ийг 3.1-ийн томьёогоор тооцно.

Яагаад 1 ms-ийн алхам? C хэрэгжилт нь slice-аар (10 ms хүртэл) нэг дор үсэрдэг боловч симулятор алхам бүрийг `log`-д хадгалдаг тул (а) slider-аар ямар ч цагт зогсож төлөвийг харж болно, (б) preemptive алгоритмуудад таслалтыг яг тухайн мөчид шалгах боломжтой. Хоёр хэрэгжилтийн *алгоритмын логик* ялгаатай (C: slice-аар үсэрдэг, холбоост жагсаалт; JS: 1 ms-ээр алхдаг, массив) тул тэдгээрийн үр дүн таарах нь бие даасан баталгаа болдог.

6.3 Gantt chart-ыг log-оос зурах

Gantt chart нь `log`-ийг *зөвхөн унших* замаар `segsOf(log, upto)` функцээр зурагдана: дараалсан ижил `run`-тай алхмуудыг нэг мөр болгон нэгтгэж (from, end) сегмент үүсгэнэ. Slider-ийн утга t өөрчлөгдөх бүрт $[0, t)$ хэсгийн сегментүүдийг дахин зурдаг тул Gantt, метрик, pseudo-code-ийн тодруулалт, ready queue бүгд ижил `log`-оос гардаг учир хоорондоо зөрөх боломжгүй.

6.4 «C-ийн гаралт оруулах» — UI ба C-ийн холбоо

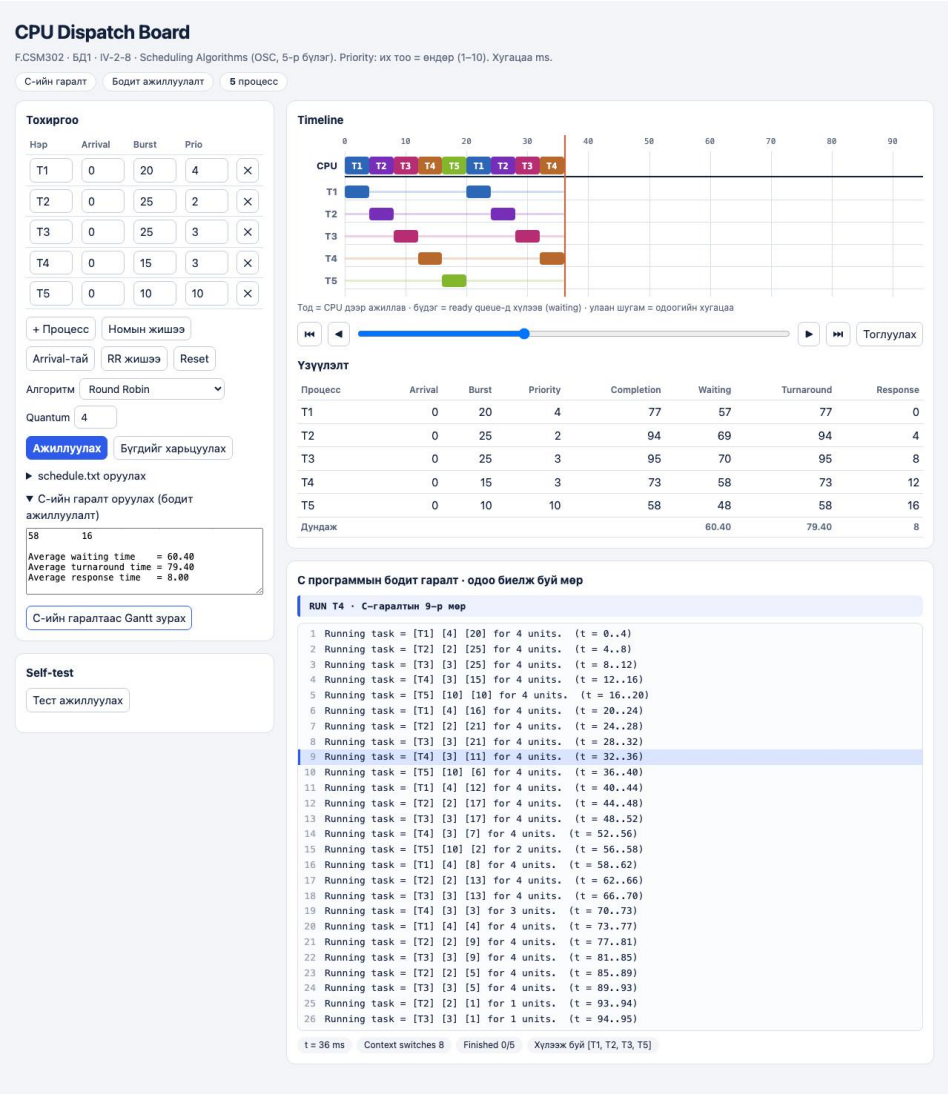
Симулятор нь C програмын хэвлэсэн мөрүүдийг (`Running task = [T1] [4] [20] for 20 units. (t = 0..20)`) regular expression-ээр задалж `parseC()` Gantt зурдаг. `parseC` нь зурахаасаа өмнө гаралтыг шалгана:

1. мөр бүрийн эхлэх цаг нь өмнөх мөрийн төгсгөлтэй тэнцүү (хугацаа тасралтгүй) ба end − from = len;
2. task бүрийн нийт ажилласан хугацаа = анхны burst (task-ийн эхний мөрийн `[remaining]`). Бүрэн бус эсвэл өөр програмын гаралтыг ингэж татгалзана.

Дараа нь waiting, turnaround, response-ийг *өөрөө дахин тооцож* C-ийн хэвлэсэн дундажтай харьцуулах боломжийг олгоно (8.6-р хэсэг). Тиймээс энэ нь зөвхөн «зураг» биш, C-ийн үр дүнг шалгах хэрэгсэл юм.

6.5 Анхаарах зүйл

Симуляторын горимд Code Viewer-т харагдах код нь алгоритмын гогцооны **pseudo-code** бөгөөд бодит `.c` файл биш (гол ялгаа: `pickNextTask` ба таслалтын нөхцөл). «C-ийн гаралт оруулах» горимд Code Viewer нь C програмын бодит гаралтын мөрүүдийг харуулна. Бодит C код `src/c/`-д байна.



Зураг 6.2. С-ийн бодит гаралтыг (RR, quantum 4) UI-д ачаалсан байдал, $t = 36$ ms: Gantt, метрик, С-гаралтын 9-р мөр тодорсон.

7. Туршилтын арга

Энэ бүлэгт «хэрхэн шалгасан бэ» гэдгийг тодорхой бичнэ: ямар орчин, ямар өгөгдөл, хүлээгдэх үр дүнг яаж гаргасан, ямар төрлийн шалгалт хийсэн.

7.1 Туршилтын орчин

Орчин 1 — баг гараар шалгасан (macOS): macOS, Terminal (zsh), gcc (Apple clang), Safari. `make` анхааруулгагүй хөрвүүлэгдэж, `make test` 18/18 давсан; C демо ба вэб демог энэ орчинд бичлэгт буулгасан. **Орчин 2 — автомат шалгалт (Linux):** gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0; Node.js v22.22.2; Linux 6.18.44-fc-v153 x86_64; Chromium 141.0.7390.37. Симулятор Chromium-д Playwright-аар автоматаар туршигдсан (зарим screenshot ингэж авагдсан).

7.2 Туршилтын өгөгдөл

1. `tests/schedule.txt` — номын жишээ: T1,4,20; T2,2,25; T3,3,25; T4,3,15; T5,10,10 (Хүснэгт 3.1).
2. `tests/schedule_ties.txt` — тэнцүү priority-той гурван task (A, B, C: priority 5; burst 12, 7, 3) ба нэг бага priority-той D (priority 1, burst 9). Зорилго: тэнцүү priority-ийн ээлж, tie-breaking-ийг шалгах.
3. Алдаатай оролтууд: `bad_priority`, `bad_burst`, `bad_format`, `bad_empty`.

Хоёр дахь файлыг ялгаатайгаар сонгосон шалтгаан: номын жишээнд тэнцүү priority нь зөвхөн T3, T4 хос бөгөөд нэг л удаа tie шалгагдана. `schedule_ties` нь гурван task-ийн ээлж (A → B → C → A…) ба бага priority-той D-ийн хүлээлтийг нэгэн зэрэг шалгадаг.

7.3 Хүлээгдэх үр дүнг хэрхэн гаргасан бэ?

Тестийн үнэлгээ нь «програмын гаралтыг өөрөөр нь шалгах» гэсэн тойрог логикт орохгүйн тулд хүлээгдэх утгыг *C-ээс бие даан* гаргасан:

1. Эхлээд **гараар** бодсон (3-р бүлэг): жишээ нь FCFS-ийн waiting $= 0,\ 20,\ 45,\ 70,\ 85 \Rightarrow 44$.
2. `tests/expected.txt`-ийн утгуудыг C-ээс тусдаа бичигдсэн **JavaScript симуляторoор** тооцсон.
3. Нэмэлтээр тайлан бичих үед гурав дахь, бие даасан **Python-ийн товч загвар**-аар (алгоритмын дүрмийг 10 мөр кодоор дахин бичсэн) бүх хослолын дундаж үзүүлэлтийг дахин тооцсон. Гурван эх сурвалж (гараар, JS, Python) C-ийн гаралттай тулгасан бүх

хослолд таарсан. Python-ийн шалгалтыг `tests/reference_check.py`-ээр давтан ажиллуулж болно.

7.4 Шалгалтын төрлүүд

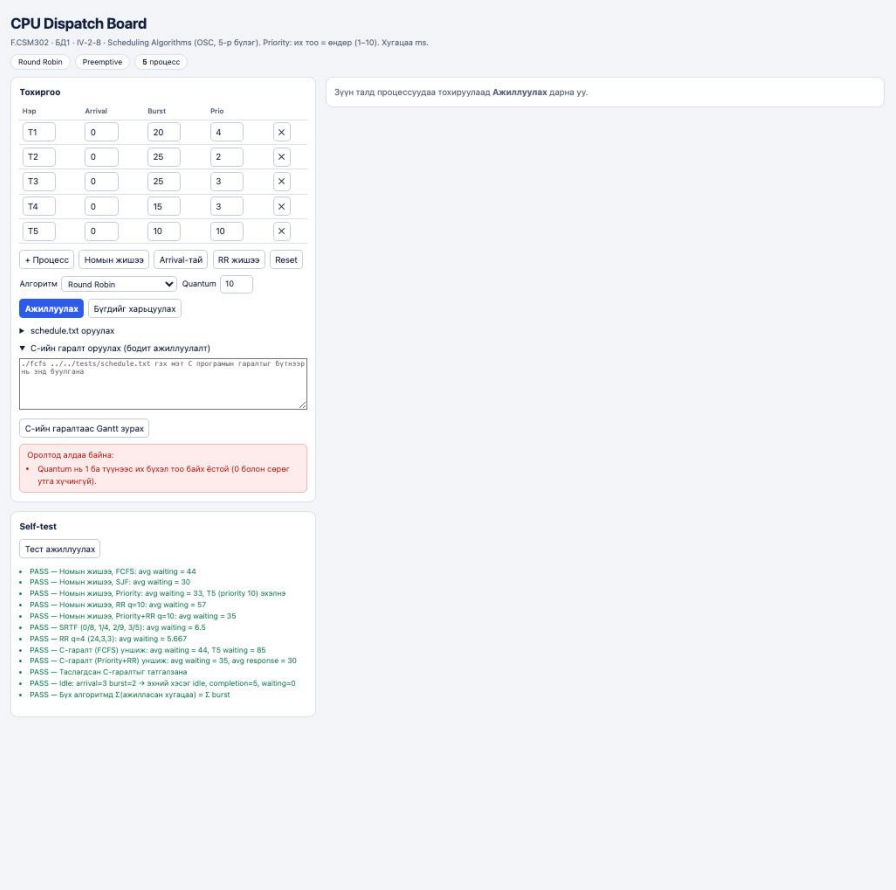
Хүснэгт 7.1. `make test`-ийн 18 шалгалт

Төрөл	**Тоо**	**Агуулга**
Алгоритм × өгөгдөл	10	Номын жишээ: FCFS, SJF, Priority, RR ($q = 10$), RR ($q = 4$), Priority+RR ($q = 10$), Priority+RR ($q = 4$); `schedule_ties`: Priority+RR ($q = 4$), Priority, SJF. Дундаж waiting/turnaround/response-ийг `expected.txt`-тэй яг (2 орон) тулгана.
Алдаатай файл	4	`bad_priority`, `bad_burst`, `bad_format`, `bad_empty`: exit code $\neq 0$ ба `stderr`-д мэдээлэл
Буруу quantum	3	0, -3, `abc`
Байхгүй файл	1	`missing.txt`
Нийт	**18**	

Нэмэлт шалгалтууд (`make test`-д ороогүй, тайланд үр дүнг нь харуулсан): (а) quantum $1 \ldots 30$-ийн эргэлт (8.4), (б) атомар tid-ийн thread-ийн туршилт (8.7), (в) симуляторын self-test 12 шалгалт ба C-гаралт уншигчийн тулгалт (8.6).

7.5 Хязгаарлалт: тестийн хамрах хүрээ

Тестүүд нь дундаж утгуудыг шалгадаг, task бүрийн waiting-ийг (8-р бүлгийн Хүснэгт 8.2) автоматаар шалгадаггүй; тэдгээрийг бид гараар бодсонтой нэг удаа тулгасан. Мөн random (санамсаргүй) өгөгдлөөр олон удаа туршиж C ба JS-ийг харьцуулах property-based тест хийгээгүй — энэ нь цаашдын сайжруулалт (9.6).



Зураг 7.1. Симулятор доторх self-test — бүх шалгалт PASS.

8. Үр дүн

Энэ бүлэгт эхлээд C хэрэгжилтийн бодит гаралтыг 3-р бүлгийн гараар бодсон үр дүнтэй тулгаж, дараа нь алгоритмуудыг харьцуулж, quantum-ын нөлөө, алдаатай оролт, UI-С тулгалт, атомар tid-ийн туршилтыг үзүүлнэ.

8.1 Алгоритм тус бүрийн үр дүн (номын жишээ)

8.1.1 FCFS

C хэрэгжилтийн бодит гаралт (`./fcfs tests/schedule.txt`):

```
Running task = [T1] [4] [20] for 20 units.  (t = 0..20)
Running task = [T2] [2] [25] for 25 units.  (t = 20..45)
Running task = [T3] [3] [25] for 25 units.  (t = 45..70)
Running task = [T4] [3] [15] for 15 units.  (t = 70..85)
Running task = [T5] [10] [10] for 10 units.  (t = 85..95)

Task     tid  Pri   Burst   Waiting  Turnaround   Response
T1         1    4     20         0          20          0
T2         2    2     25        20          45         20
T3         3    3     25        45          70         45
T4         4    3     15        70          85         70
T5         5   10     10        85          95         85

Average waiting time   = 44.00
Average turnaround time = 63.00
Average response time   = 44.00
```

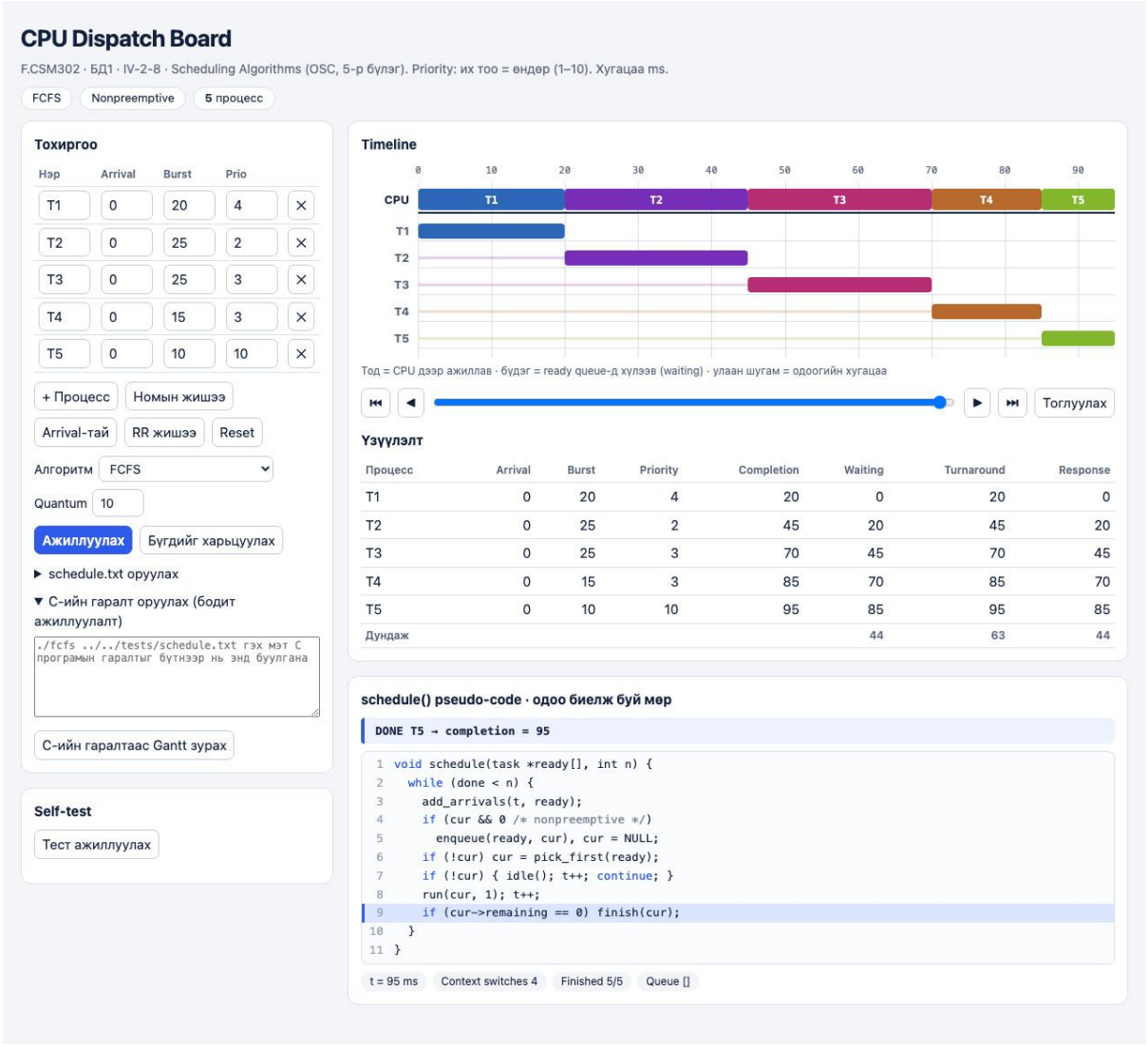
Гаралт 3.3-д гараар бодсон Gantt (T1 0–20, T2 20–45, …, T5 85–95) болон дундаж 44-тэй *яг таарсан*. Зураг 8.1-д симуляторын харагдац.

8.1.2 SJF

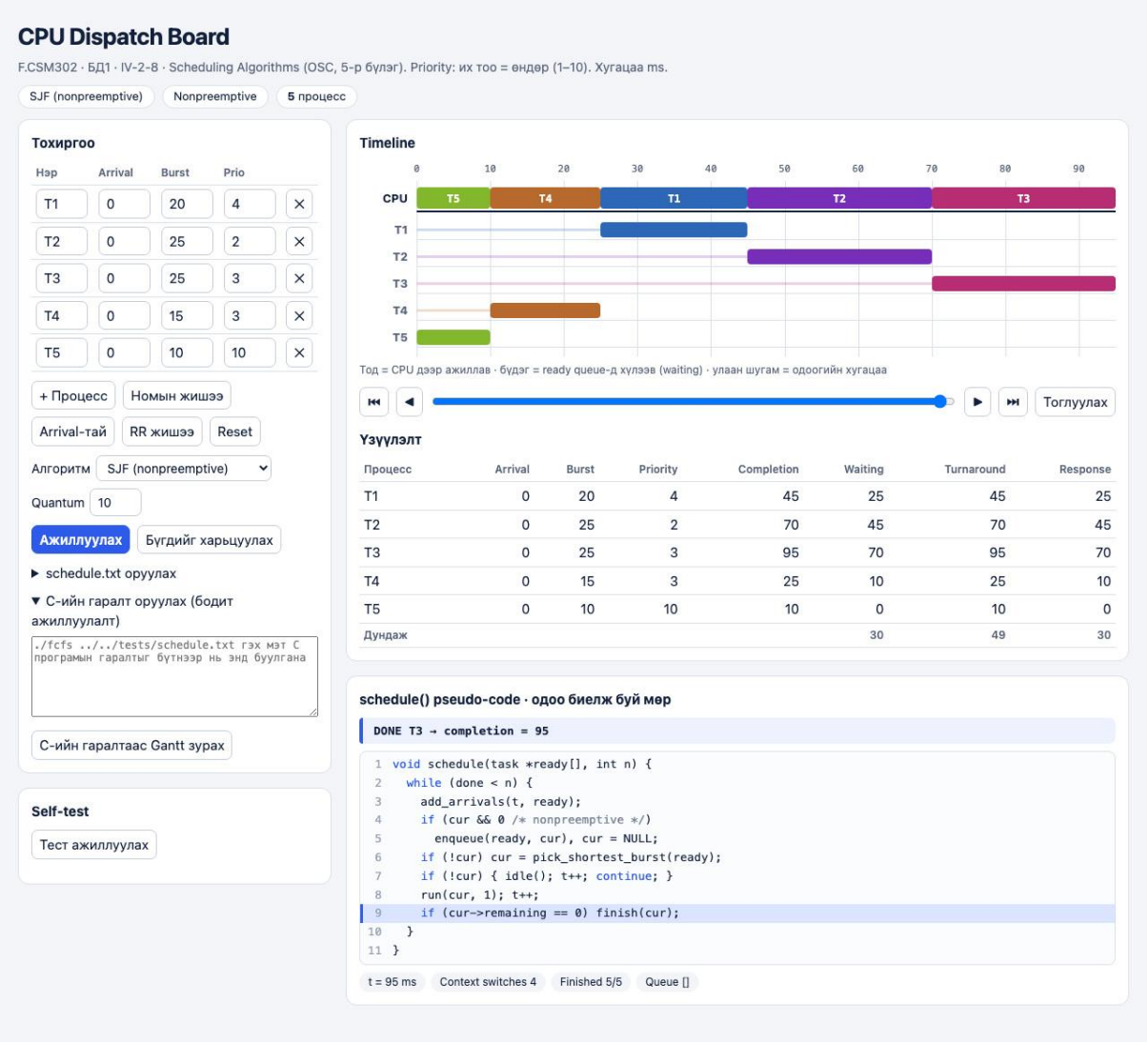
Бодит гаралтаас дараалал T5, T4, T1, T2, T3 (T2, T3 нь burst 25-аар тэнцүү; T2 файлд эхэнд байсан тул түрүүлсэн). Дундаж waiting 30.00, turnaround 49.00, response 30.00 — гараар бодсонтой (3.4) таарсан.

8.1.3 Priority

Дараалал T5 (10), T1 (4), T3 (3), T4 (3), T2 (2). Дундаж waiting 33.00, turnaround 52.00, response 33.00 — гараар бодсонтой (3.5) таарсан. T2 (priority 2) нь хамгийн сүүлд 70 ms хүлээсэн.



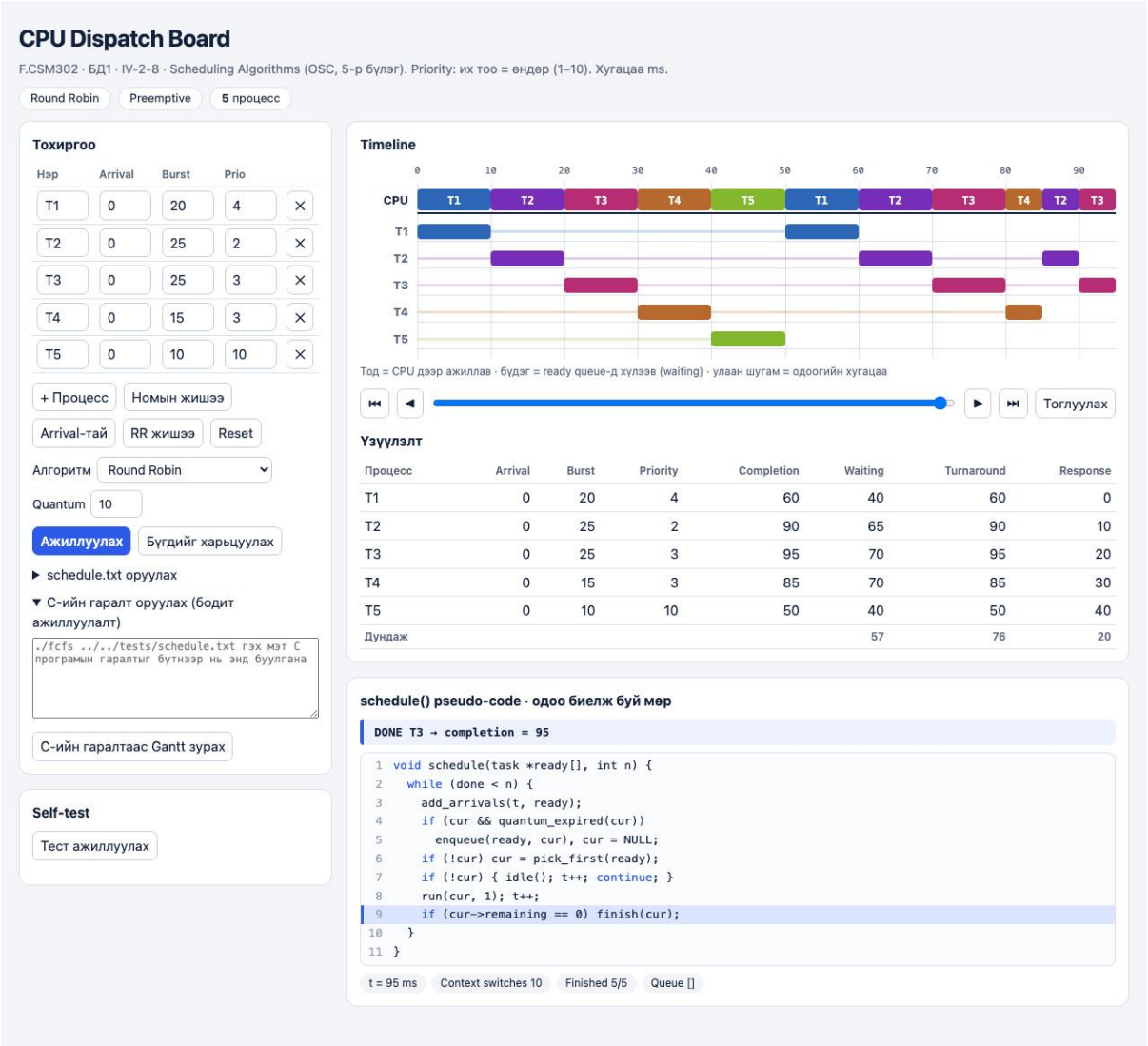
Зураг 8.1. FCFS — богино T5 нь хамгийн сүүлд (85 ms хүлээсэн). Бүдэг шугам нь хүлээсэн хугацаа.



Зураг 8.2. SJF — богино task-ууд эхэлж, дундаж waiting хамгийн бага (30).

8.1.4 Round-Robin ($q = 10$)

Дундаж waiting 57.00, turnaround 76.00, response 20.00. Нийт 11 slice, context switch 10. Зураг 8.3-д харагдаж буй ээлж 3.6-ийн гараар бодсонтой таарна.



Зураг 8.3. Round-Robin ($q = 10$) — task бүр 10 ms-ээр ээлжилнэ; context switch 10.

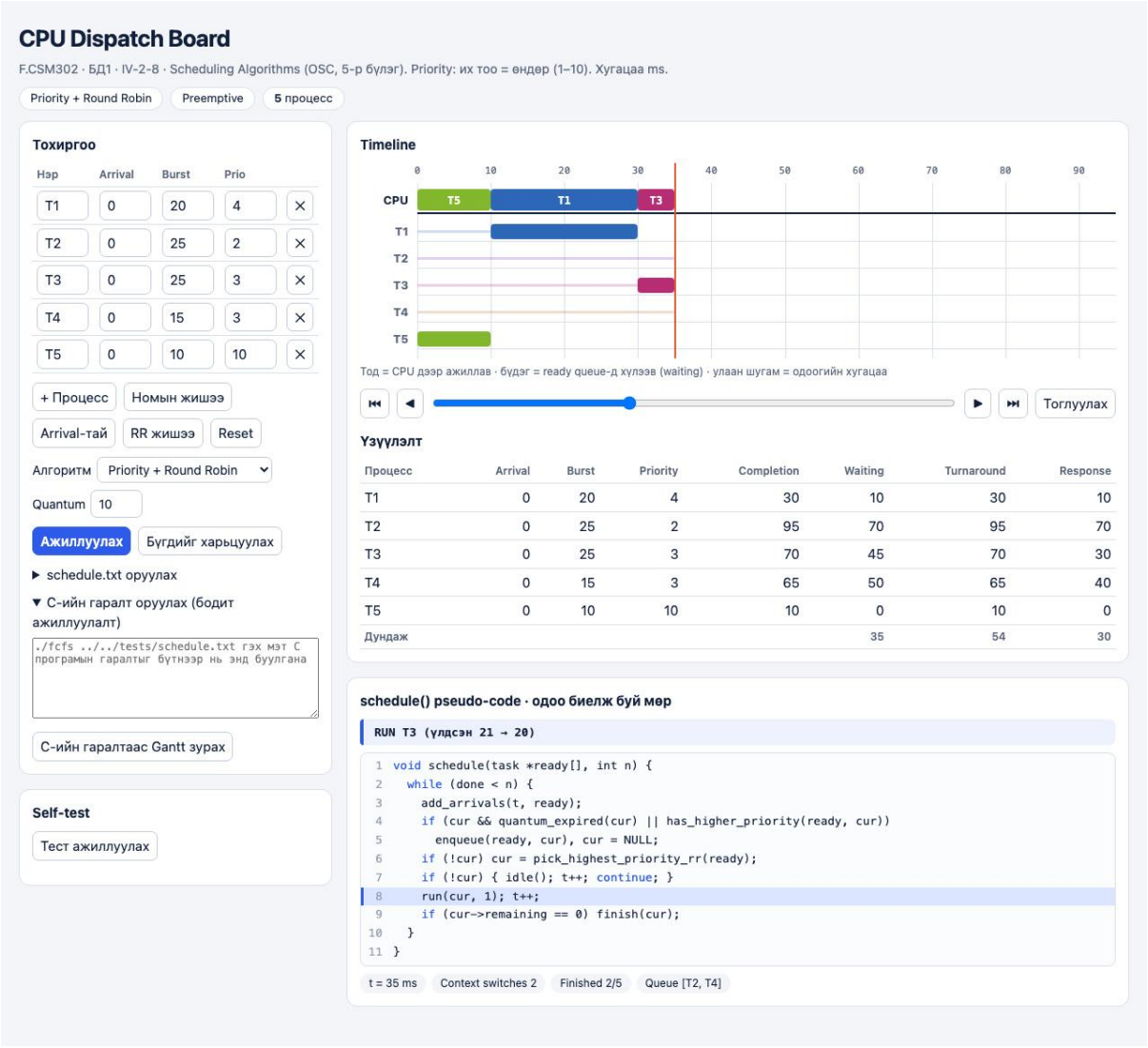
8.1.5 Priority + Round-Robin ($q = 10$)

Бодит гаралт — T3 ба T4 (priority 3) ээлжилж, T2 (priority 2) хамгийн сүүлд ажиллана:

```
Running task = [T5] [10] [10] for 10 units.  (t = 0..10)
Running task = [T1] [4] [20] for 10 units.  (t = 10..20)
Running task = [T1] [4] [10] for 10 units.  (t = 20..30)
Running task = [T3] [3] [25] for 10 units.  (t = 30..40)
Running task = [T4] [3] [15] for 10 units.  (t = 40..50)
```

```
Running task = [T3] [3] [15] for 10 units.  (t = 50..60)
Running task = [T4] [3] [5] for 5 units.  (t = 60..65)
Running task = [T3] [3] [5] for 5 units.  (t = 65..70)
Running task = [T2] [2] [25] for 10 units.  (t = 70..80)
Running task = [T2] [2] [15] for 10 units.  (t = 80..90)
Running task = [T2] [2] [5] for 5 units.  (t = 90..95)
```

Дундаж waiting 35.00, turnaround 54.00, response 30.00. Гараар бодсон Gantt (3.7)-тэй бүх slice таарсан.



Зураг 8.4. Priority + RR, $t = 35$ алхамд — T5, T1 дууссан; T3 ажиллаж байна; pseudo-code-ийн 8-р мөр тодорсон.

8.2 Нэгтгэсэн үр дүн

Хүснэгт 8.1. Алгоритм тус бүрийн дундаж үзүүлэлт (ms) ба context switch-ийн тоо

Алгоритм	Avg waiting	Avg turnaround	Avg response	Context switch
FCFS	44.00	63.00	44.00	4
SJF	**30.00**	**49.00**	30.00	4
Priority	33.00	52.00	33.00	4
Round Robin ($q = 10$)	57.00	76.00	**20.00**	10
Priority + RR ($q = 10$)	35.00	54.00	30.00	10

Context switch-ийн тоо = slice-ийн тоо -1. Nonpreemptive алгоритмуудад slice = $n = 5$ тул 4.

Хүснэгт 8.2. Task бүрийн waiting time (ms)

Task (priority, burst)	FCFS	SJF	Priority	Round Robin ($q = 10$)	Priority + RR ($q = 10$)
T1 (4, 20)	0	25	10	40	10
T2 (2, 25)	20	45	70	65	70
T3 (3, 25)	45	70	30	70	45
T4 (3, 15)	70	10	55	70	50
T5 (10, 10)	85	0	0	40	0

Нүд бүрт «response / completion» гэж бичсэн. Энэ хүснэгтээс turnaround = completion, waiting = completion − burst гэж шууд гаргаж болох тул Хүснэгт 8.2-ийг шалгах боломжтой (жишээ нь Priority+RR-ийн T3: $70 - 25 = 45$).

8.3 Тэнцүү priority-ийн туршилт (`schedule_ties.txt`)

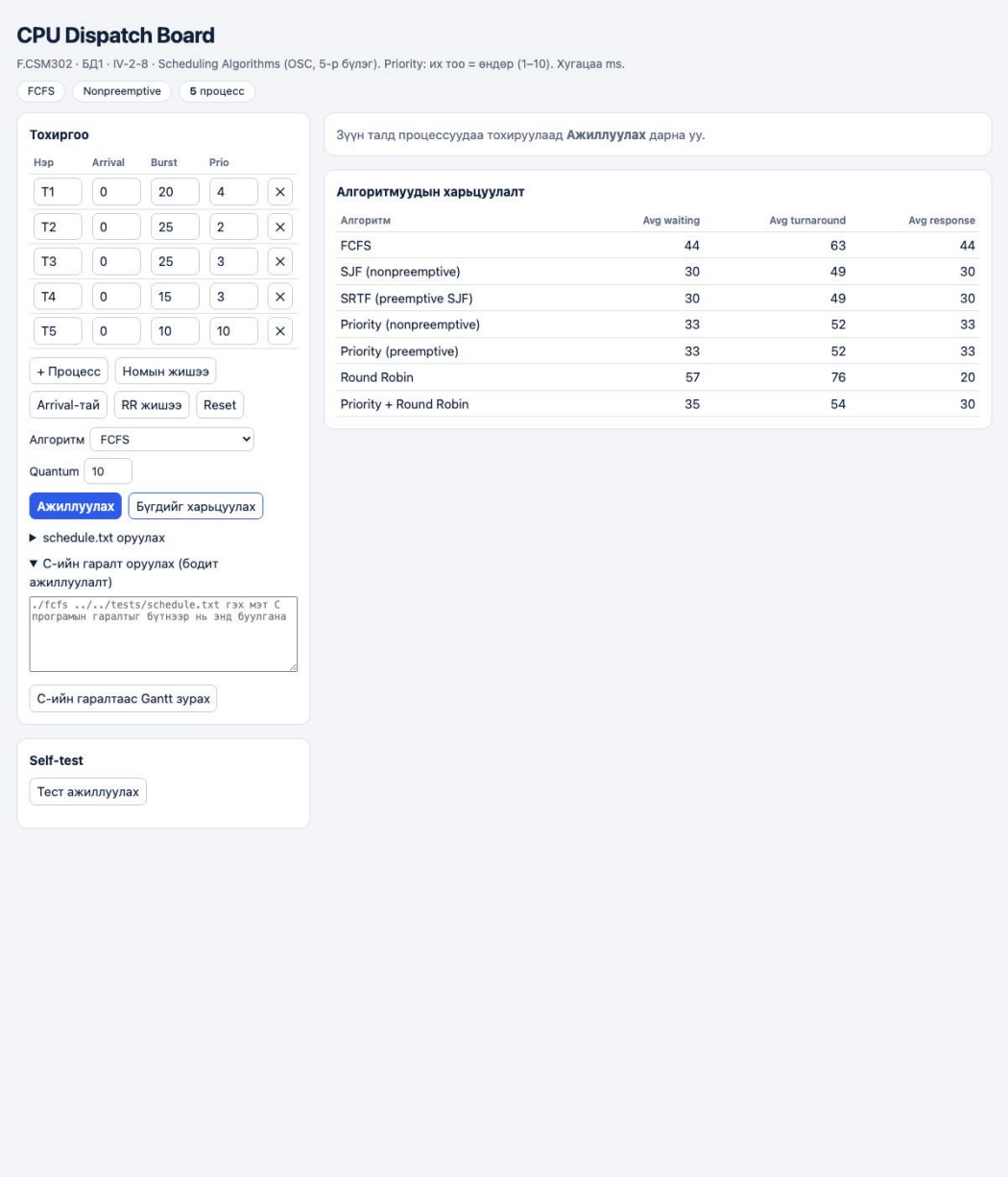
Task: A (5, 12), B (5, 7), C (5, 3), D (1, 9). Priority+RR, $q = 4$. Програмын гаралтаас Gantt:



A, B, C хоорондоо ээлжилж, D нь тэдгээр бүгд дууссаны дараа л (22-р ms) эхэлсэн: priority-ийн дараа RR ажилласны нотолгоо. Waiting: A = $22-12 = 10$, B = $18-7 = 11$, C = $11-3 = 8$,

Хүснэгт 8.3. Task бүрийн анх CPU авсан цаг (response) ба дууссан цаг (completion), ms

Task	FCFS	SJF	Priority	RR ($q = 10$)	Priority+RR
T1	0 / 20	25 / 45	10 / 30	0 / 60	10 / 30
T2	20 / 45	45 / 70	70 / 95	10 / 90	70 / 95
T3	45 / 70	70 / 95	30 / 55	20 / 95	30 / 70
T4	70 / 85	10 / 25	55 / 70	30 / 85	40 / 65
T5	85 / 95	0 / 10	0 / 10	40 / 50	0 / 10



Зураг 8.5. «Бүгдийг харьцуулах» — 7 алгоритмын дундаж (симулятор).

D = 31 − 9 = 22; дундаж = $\frac{51}{4}$ = 12.75 — `expected.txt`-тэй таарсан. Мөн тодорхой tie-breaking: ижил өгөгдлийг энгийн Priority-д ажиллуулахад A, B, C файлын дарааллаар (A → B → C → D) ажиллаж дундаж waiting 13.25 болсон.

8.4 Quantum-ын нөлөө

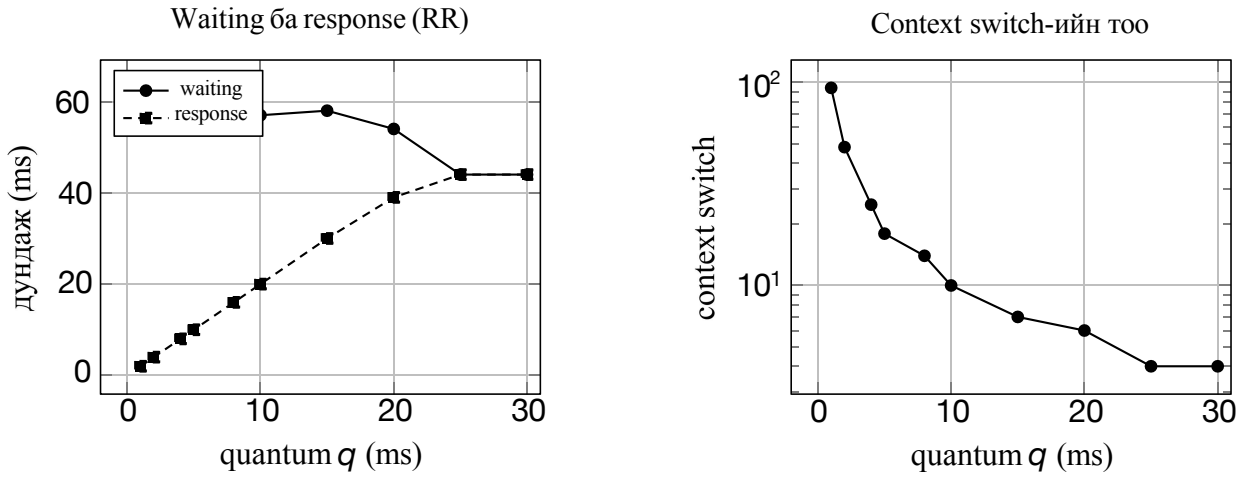
Quantum-ыг 1, 2, 4, 5, 8, 10, 15, 20, 25, 30 гэж өөрчилж, номын өгөгдөл дээр RR ба Priority+RR-ийг ажиллуулав (Хүснэгт 8.4, Зураг 8.6).

Хүснэгт 8.4. Quantum-аас хамаарсан дундаж үзүүлэлт (ms) ба context switch

	Round-Robin			**Priority + RR**		
q	**waiting**	**response**	**c.s.**	**waiting**	**response**	**c.s.**
1	59.4	2.0	94	34.0	28.2	94
2	59.6	4.0	48	34.2	28.4	48
4	60.4	8.0	25	34.2	28.8	25
5	57.0	10.0	18	34.0	29.0	18
8	63.0	16.0	14	34.2	29.6	14
10	57.0	20.0	10	35.0	30.0	10
15	58.0	30.0	7	34.0	31.0	7
20	54.0	39.0	6	35.0	32.0	6
25	44.0	44.0	4	33.0	33.0	4
30	44.0	44.0	4	33.0	33.0	4

Ажиглалт.

1. **Response нь quantum-тай шууд пропорционал:** $\bar{r} = \frac{(n-1)q}{2}$ ($q \le 10$ үед бүх burst $\ge q$ тул 3.6-ийн томьёо яг биелнэ: $q = 1 \to 2.0, q = 4 \to 8.0, q = 10 \to 20.0$).
2. **Context switch нь quantum багасахад огцом өснө:** $q = 10$ үед 10 бол $q = 1$ үед 94. Бидний загвар switch-ийн зардлыг 0 гэж үздэг тул энэ зардал waiting-д орохгүй; бодит системд $q = 1$ нь маш өндөр overhead үүсгэнэ.
3. **Waiting нь quantum-тай хамааралтай ч монотон биш:** $q = 5$ үед 57.0, харин $q = 4$ үед 60.4, $q = 8$ үед 63.0. Боломжит тайлбар (бид үүнийг тусад нь нотлоогүй, таамаглал): task-уудын burst (10, 15, 20, 25) quantum-д хэр «таардаг»-аас хамаарна. Жишээ нь $q = 5$ үед бүх burst 5-ын үржвэр тул task бүр бүтэн slice-аар дуусдаг; $q = 8$ үед $25 = 3 \cdot 8 + 1$ мэт жижиг үлдэгдэл slice үүсэж, task-уудын дуусах дараалал өөрчлөгддөг. Хэрхэн ч байсан,
   «quantum багасгавал waiting сайжирна» гэж ерөнхийлж болохгүй гэдгийг тоо харуулж байна.



Зураг 8.6. Quantum-ын нөлөө (RR, номын өгөгдөл). Зүүн: waiting ба response; баруун: context switch (логарифм тэнхлэг).

4. **Хязгаарын тохиолдол (корректнессийн нотолгоо):** $q \geq 25$ (хамгийн урт burst) үед RR нь FCFS-тэй (44.00 / 63.00 / 44.00), Priority+RR нь Priority-той (33.00 / 52.00 / 33.00) *яг тэнцүү* болсон. Энэ нь онолын таамаглалтай нийцэж, хэрэгжилт зөв гэдгийг батлах бие даасан шалгалт болсон.

8.5 Буруу оролт

Алдаатай оролтод C хэрэгжилтийн хариу:

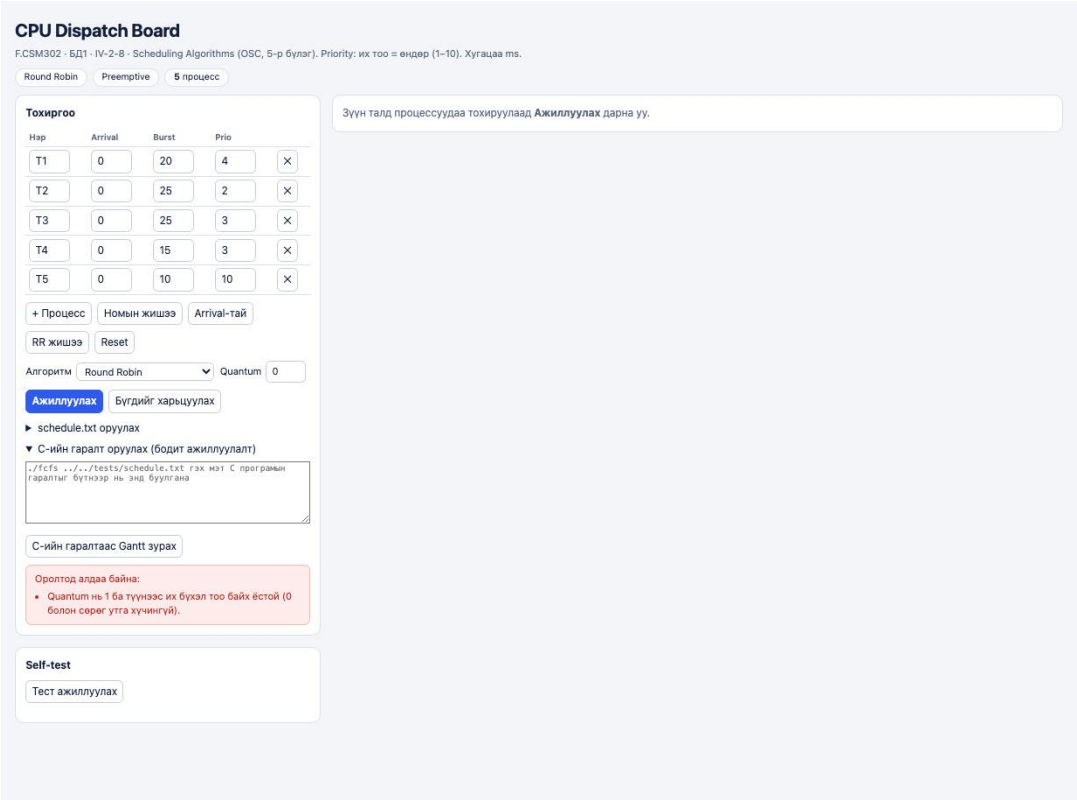
```
Error (../../tests/bad_priority.txt, line 2): priority 11 is outside 1..10.
Error: quantum must be an integer from 1 to 1000 (got "0").
```

Хоёр тохиолдолд аль аль нь ажиллуулалт хийгдэхгүй, exit code $\neq 0$. Симулятор дээр мөн адил (Зураг 8.7).

8.6 UI ба C хэрэгжилтийн тулгалт

C програмын 6 гаралтыг (FCFS, SJF, Priority, RR $q = 10$, RR $q = 4$, Priority+RR) симуляторт буулгаж, симуляторын *өөрөө дахин тооцсон* дундаж waiting / turnaround / response-ийг C-ийн хэвлэсэн дундажтай автоматаар тулгав — **6/6 таарсан (нэмэлтээр schedule_ties.txt-ийн Priority+RR q = 4 гаралт ч таарч, нийт 7/7)**.

Нэмэлтээр симуляторын *өөрийн алгоритмын цөм* (`simulate()`) нь C-тэй адил дунджийг `schedule_ties.txt` дээр ч (Priority+RR $q = 4$: 12.75 / 20.50 / 8.50) гаргасан.



Зураг 8.7. Буруу оролт (quantum = 0) — ойлгомжтой алдааны мэдээлэл, ажиллуулалт хийгдэхгүй.

Хүснэгт 8.5. UI-ийн тооцсон дундаж ба С-ийн хэвлэсэн дундажийн тулгалт

С гаралт	**UI (waiting / turnaround / response)**	**С-ийн хэвлэсэн**
FCFS	44 / 63 / 44	44.00 / 63.00 / 44.00
SJF	30 / 49 / 30	30.00 / 49.00 / 30.00
Priority	33 / 52 / 33	33.00 / 52.00 / 33.00
RR $q = 10$	57 / 76 / 20	57.00 / 76.00 / 20.00
RR $q = 4$	60.4 / 79.4 / 8	60.40 / 79.40 / 8.00
Priority + RR	35 / 54 / 30	35.00 / 54.00 / 30.00

8.7 Атомар tid-ийн туршилт

`tests/tid_race_test.c`: 8 thread тус бүр тоолуурыг 200 000 удаа нэмэгдүүлнэ; хүлээгдэх эцсийн утга 1 600 001. Нэг тоолуур энгийн `x = x + 1`-ээр, нөгөө нь атомар builtin-ээр нэмэгдэнэ. Дараах нь нэг ажиллуулалтын гаралт:

```
expected counter value : 1600001
plain  x = x + 1       : 1000001  (lost updates: 600000)
__sync_fetch_and_add   : 1600001  (lost updates: 0)
```

Тестийг 10 удаа давтахад (нэг цөмтэй Linux орчин): атомар хувилбар **10/10 удаа** алдаагүй (lost updates = 0); энгийн `x = x + 1` хувилбар **5/10 удаа** update алдсан (200 000–1 000 000 ширхэг), үлдсэн 5 удаад thread-үүд таслагдалгүй дараалан ажилласан тул алдаа гараагүй. Race condition нь *заавал* гардаггүй, харин санамсаргүй гардаг тул тэр нь аюултай: туршилтад
«тохиолдлоор» давж болдог алдаа бодит олон цөмт машинд илүү олон тохиолдох болно. Иймээс номын шаардсан атомар олголт үндэслэлтэй.

9. Хэлэлцүүлэг

9.1 Онолын таамаглал ба бодит үр дүн

Хүснэгт 9.1. Онолын таамаглал ба манай үр дүн

Онолын таамаглал	**Манай үр дүн**	**Дүгнэлт**
SJF нь дундаж waiting-ийг хамгийн бага болгоно (бүгд $t = 0$-д ирэхэд)	SJF 30.00; бусад нь 33.00–57.00	Нийцсэн
FCFS-д convoy effect	Хамгийн богино T5 85 ms хүлээсэн	Нийцсэн
Priority-д бага priority-той task өлсөнө	T2 (priority 2) хамгийн сүүлд, waiting 70	Нийцсэн (8-р бүлэг, 9.3)
RR нь response-д хамгийн сайн	RR response 20.00 (бусад ≥ 30)	Нийцсэн
RR $q \to \infty$ үед FCFS болно	$q \geq 25$ үед 44.00 / 63.00 / 44.00	Нийцсэн
Priority+RR $q \to \infty$ үед Priority болно	$q \geq 25$ үед 33.00 / 52.00 / 33.00	Нийцсэн
Quantum багасахад waiting муудна	Үргэлж биш: $q = 5$-д 57.0, $q = 8$-д 63.0	Онолыг тодруулсан (8.4)

9.2 Алгоритм тус бүрийн дүн шинжилгээ

- **FCFS** (waiting 44.00). Хамгийн энгийн, context switch хамгийн цөөн (4). Гэхдээ дундаж нь task-ийн дарааллаас шууд хамаарна: хэрэв T5-г эхэнд нь тавивал дундаж waiting буурна (T5, T1, T3, T4, T2 дарааллаар $\frac{0+10+30+55+70}{5} = 33$). Энэ нь FCFS «шударга» боловч «үр дүнтэй» биш гэдгийг харуулна.
- **SJF** (30.00). Онолын хувьд оновчтой. Гэхдээ бидний жишээнд бүх burst-ийг мэдэж байгаа гэж үзсэн; бодит системд дараагийн burst-ийг урьдчилан мэдэхгүй учир экспоненциал дунджаар таамагладаг (номын 5.3.2). Энэ ажилд таамаглал хэрэгжүүлээгүй.
- **Priority** (33.00). SJF-ээс 3 ms-ээр л дор. Priority-г «burst-ийн урвуу» гэж сонговол SJF болдог тул энэ нь санамсаргүй биш. Харин жишээнд priority нь burst-тэй хамааралгүй тул тийм ойр гарсан нь тохиолдол. Чухал task (T5) шууд эхэлсэн боловч T2 70 ms хүлээв.

- **Round-Robin** (waiting 57.00, turnaround 76.00 — хамгийн муу; response 20.00 — хамгийн сайн). Богино task T5 (10 ms) 40 ms-д эхэлж 50-д дууссан нь FCFS-ийн 85-аас сайн; харин урт task-ууд (T2, T3) олон удаа таслагдаж 90–95-д дууссан. Нийт ажил адил (95 ms) боловч task бүр хэсэг хэсгээрээ дууссан тул дундаж turnaround өндөр: RR нь «бүгдийг нэгэн зэрэг явуулж, бүгдийг сүүлд нь дуусгадаг».
- **Priority + RR** (35.00). Priority-ийн давуу талыг (чухал нь түрүүлнэ) хадгалж, тэнцүү priority-той T3, T4-ийн response-ийг сайжруулсан. Харин priority-ууд хоорондын starvation үлдэнэ (T2 хамгийн сүүлд).

9.3 Starvation-ийн туршилт

Номын жишээнд бүх task $t = 0$-д ирдэг тул starvation «түр хойшлох» хэмжээнд л харагддаг (T2 70 ms). Үнэхээр өлсөх нөхцөлийг үзүүлэхийн тулд arrival дэмждэг симулятор дээр дараах туршилтыг хийв: бага priority-той L (priority 1, burst 5, $t = 0$) ба өндөр priority-той H1…H6 (priority 9, burst 10), H_k нь $t = 10(k - 1)$ мөчид ирнэ. H task-ууд 10 ms тутамд ирж, тус

Хүснэгт 9.2. L task-ийн хүлээлт: FCFS ба Priority

Алгоритм	L эхэлсэн цаг	L-ийн waiting	H task-уудын waiting
FCFS	0	0	5 (бүгд)
Priority (nonpreemptive)	60	**60**	0 (бүгд)
Priority + RR ($q = 10$)	60	**60**	0 (бүгд)

бүр 10 ms ажилладаг тул CPU хэзээ ч сул болохгүй, L нь $t = 60$ хүртэл (H6 дуустал) огт ажиллаагүй. Хэрэв H task-ууд хязгааргүй ирсээр байвал L хэзээ ч ажиллахгүй, өөрөөр хэлбэл *бодитоор өлсөнө*. Priority+RR нь үүнийг засаагүй, учир нь RR нь зөвхөн *тэнцүү* priority-ийн дунд ээлж олгодог. Шийдэл болох aging (хүлээсэн хугацаа тутамд priority нэмэх) энэ ажилд хэрэгжээгүй. Энэ туршилт нь C хэрэгжилт биш, симулятор дээр хийгдсэн гэдгийг тэмдэглэе.

9.4 Үзүүлэлтүүдийн солилцоо (trade-off)

Нэг алгоритм бүх үзүүлэлтэд хамгийн сайн байж чадахгүй. Хүснэгт 8.1-аас: SJF нь waiting ба turnaround-д, RR нь response-д хамгийн сайн; гэтэл RR нь waiting ба turnaround-д хамгийн муу. Иймд алгоритмын сонголт нь системийн зорилгоос хамаарна: интерактив (response чухал) системд RR төрлийн, багц боловсруулалтад (turnaround чухал) SJF төрлийн, чухал ажлыг баталгаажуулах шаардлагатай системд Priority төрлийн алгоритм тохирно.

9.5 Хязгаарлалт

1. С хувилбар номын нөхцөлөөр бүх task $t = 0$-д ирнэ гэж үзнэ; arrival, preemption (SRTF, preemptive Priority)-ыг зөвхөн симулятор дэмжинэ.
2. Context switch-ийн зардлыг 0 ms гэж үзсэн; тоог нь тоолсон боловч хугацаанд нөлөөлүүлээгүй.
3. I/O болон блоклогдох төлөвгүй; task нь зөвхөн нэг CPU burst-тэй. Бодит процесс CPU–I/O burst-ийн мөчлөгтэй.
4. Нэг CPU ба нэг thread. Олон цөмт scheduling (load balancing, processor affinity) хэрэгжээгүй. Атомар tid-ийн race-ийг тусдаа туршилтаар л үзсэн; `all_tasks[]` бүртгэл олон thread-д аюулгүй биш (5.2.2).
5. Task нэрийн давхардлыг С хувилбар шалгадаггүй (симулятор шалгана). Нэр давхардвал `report()` хоёр мөр хэвлэнэ, харин `delete` нь заагчаар ажилладаг тул алгоритмын үр дүнд нөлөөлөхгүй.
6. Симулятор: burst ≤ 1000, ≤ 20 процесс. Симуляторын горимд Code Viewer нь pseudo-code.
7. Симуляторын «Context switches» тоолуур нь дараалсан ижил task-ийг нэг сегмент болгон нэгтгэдэг тул (жишээ нь Priority+RR-д T1 10–20, 20–30) С-ийн slice-аар тоолсон утгаас цөөн гарч болно; хэрэгжилтүүдийн дундаж үзүүлэлтэд нөлөөлөхгүй.
8. Симуляторд процессын нэрийг HTML-д шууд оруулдаг тул (зөвхөн өөрийн хөтөч дээр) тусгай тэмдэгттэй нэр оруулбал дүрслэл эвдрэх боломжтой. Нийтэд нээлттэй байршуулах (Vercel) үед нэрийг escape хийх нь зүйтэй.

9.6 Цаашдын сайжруулалт

- `all_tasks[]`-ийн бүртгэлийг mutex-ээр хамгаалж, олон thread-тэй драйвер бичих.
- Aging нэмж Priority-ийн starvation-ийг арилгах; үр дүнг 9.3-ийн туршилтаар харьцуулах.
- SJF-д экспоненциал дунджаар дараагийн burst-ийг таамаглах.
- С ба JS хэрэгжилтийг санамсаргүй өгөгдлөөр олон мянга удаа харьцуулах автомат тест (property-based testing).
- `insert`-ийг tail заагчтай болгож $O(1)$ болгох.

9.7 Ажлаас сурсан зүйл

Алгоритмуудыг зөвхөн томьёогоор биш, *шийдвэрийн дүрэм*-ээр нь ойлгох нь чухал гэдгийг харсан: таван алгоритмын кодын ялгаа нь `pickNextTask()`-ийн нэг харьцуулалт ба slice-ийн урт хоёр. Тэнцүү утгын шийдвэр (< эсвэл <=) нь үр дүнг өөрчилдөг тул тайландаа тодорхой тэмдэглэх шаардлагатай болохыг мэдэв. Мөн хэрэгжилтийг гараар бодолт, бие даасан хоёр дахь хэрэгжилт (симулятор), хэмжилтээр олон талаас шалгах нь нэг тестийн давуу талаас илүү найдвартай гэдгийг ойлгов.

10. Дүгнэлт

Номын таван алгоритм (FCFS, SJF, Priority, Round-Robin, Priority with RR) болон хоёр нэмэлт даалгаврыг (атомар tid, дундаж waiting/turnaround/response) C хэл дээр номын бүтцийн дагуу (`driver`, `CPU.c` дахь `run()`, холбоост жагсаалт, `Makefile`) бүрэн хэрэгжүүлэв.

Шалгалтын гол үр дүн:

1. Номын жишээн дээр алгоритм бүрийг эхлээд гараар бодож (Gantt chart-тай), дараа нь C-ийн гаралттай тулгахад бүх дундаж ба task бүрийн хугацаа таарсан.
2. `make test`-ийн 18 шалгалт давсан; бие даасан Python загвар C-тэй 20 хослолд (2 өгөгдөл × 5 алгоритм × 2 quantum), JavaScript цөм 14 хослолд (2 өгөгдөл × 7 тохиргоо) таарсан.
3. Симулятор C-ийн бодит гаралтыг уншиж Gantt зурж, дунджийг дахин тооцоход 7/7 гаралт таарсан.
4. $q \geq 25$ үед RR нь FCFS, Priority+RR нь Priority-той тэнцэх нь онолын таамаглалтай нийцэж, хэрэгжилтийн зөв байдлыг бие даан баталсан.
5. Атомар tid-ийн thread туршилтаар энгийн тоолуур 10 удаагийн 5-д нь update алдсан бол `__sync_fetch_and_add` 10/10 удаа алдаагүй.

Харьцуулалтаас: SJF нь дундаж waiting (30.00) ба turnaround (49.00)-д оновчтой, RR нь response-д (20.00) хамгийн сайн боловч waiting (57.00)-д хамгийн муу, Priority нь starvation-ийн эрсдэлтэй (туршилтаар L task 60 ms хүлээсэн). Нэмэлт нээлт нь quantum-ыг багасгахад waiting үргэлж муудаад байдаггүй явдал ($q = 5$-д 57.0, $q = 8$-д 63.0).

Gantt chart бүхий интерактив симулятор нь параметрийг өөрчилж дахин туршихад зориулагдсан бөгөөд C хэрэгжилттэй бие даасан хоёр хэрэгжилтийн үр дүн таарсан нь зөв ажиллагааны нотолгоо болсон. Үлдсэн хязгаарлалтуудыг (олон thread-ийн бүртгэл, aging байхгүй, context switch-ийн зардал 0) 9.5-д нэрлэж, сайжруулах саналыг 9.6-д өгсөн.

Ашигласан материал

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*. Wiley, 10th ed., 2018. Chapter 5 “CPU Scheduling”, Programming Projects “Scheduling Algorithms” (P-29–P-31).

[2] “GCC, the GNU Compiler Collection.” `https://gcc.gnu.org/`.

[3] “GNU Make.” `https://www.gnu.org/software/make/`.

[4] “Chromium.” `https://www.chromium.org/`.

[5] “Playwright.” `https://playwright.dev/`.

AI ашигласан тухай

Claude (Anthropic) кодын эхний хувилбар, тест, баримт бичгийн ноорог бичихэд ашиглагдсан. Тайланг дэлгэрэнгүй болгох (бүлгүүдийг өргөтгөх, схем, гараар бодсон жишээ, quantum-ын туршилт, tid_race_test.c, Python-ийн бие даасан шалгалт нэмэх) ажилд мөн Claude-ийг ашигласан. Демо видеоны хоолойг ElevenLabs (AI дуу хоолой)-оор үүсгэсэн; screenshot-ын нэг хэсгийг Playwright-аар автоматаар авсан.

Хавсралт А. Төслийн мэдээлэл ба тестийн төлөвлөгөө

Хүснэгт А.1. Төслийн ерөнхий мэдээлэл

Талбар	**Мэдээлэл**
Хуваарилалтын код	IV-2-8
Төсөл	Scheduling Algorithms
Хэл/сан/орчин	C (gcc, make); HTML/CSS/JS
Үндсэн дүрслэл	Gantt chart
Repository	https://github.com/Munkhjin12/fcsm302_hw
Онлайн симулятор	https://fcsm302hw.vercel.app

Хүснэгт А.2. Тестийн төлөвлөгөө

Тест	**Оролт/нөхцөл**	**Хүлээгдсэн үр дүн**
T1	`tests/schedule.txt`, FCFS	avg waiting 44.00
T2	`tests/schedule.txt`, SJF	avg waiting 30.00
T3	`tests/schedule.txt`, Priority	avg waiting 33.00
T4	`tests/schedule.txt`, RR q=10	avg waiting 57.00
T5	`tests/schedule.txt`, Priority+RR q=10	avg waiting 35.00
T6	`schedule_ties.txt`, Priority+RR q=4	avg waiting 12.75
T7	`bad_priority.txt`	алдааны мэдээлэл, exit $\neq 0$
T8	`rr` … quantum 0	алдааны мэдээлэл, exit $\neq 0$
T9	C-ийн 6 гаралтыг UI-д ачаалах	UI-ийн дундаж = C-ийн дундаж
T10	`rr`, `priority_rr` $q = 1 \ldots 30$	$q \geq 25$ үед FCFS/Priority-той тэнцүү
T11	`tid_race_test` (8 thread)	Атомар тоолуур = 1 600 001

Хавсралт В. Шаардлагын биелэлт

Хүснэгтийн «Шаардлага» нь 2-р бүлгийн Хүснэгт 2.1-ийн R1–R9-тэй харгалзана.

Хүснэгт В.1. Шаардлагын биелэлт ба нотолгоо

Шаардлага	**Биелэлт ба нотолгоо**
R1: 5 алгоритм	`src/c/fcfs.c`, `sjf.c`, `priority.c`, `rr.c`, `priority_rr.c`; гараар бодолт 3-р бүлэг, бодит гаралт 8.1
R2, R3: Priority 1–10, quantum 10	`task.h` (`MIN/MAX_PRIORITY`, `QUANTUM`); `driver.c`-д шалгана; тест `bad_priority`
R4: `schedule.txt` формат	`driver.c` — «name, priority, burst»; `#` тайлбар, хоосон мөр алгасна; Хүснэгт 5.1
R5, R6, R7: driver / `CPU.c` `run()` / Makefile	`driver.c`, `CPU.c`, `Makefile` (`make fcfs`; `./fcfs schedule.txt`)
R8: Атомар tid	`tasks.c` — `__sync_fetch_and_add`; `tests/tid_race_test.c` (8.7)
R9: Дундаж метрик	`tasks.c` `report()`; Хүснэгт 8.1
Өөр өгөгдөл/параметр	`schedule.txt`, quantum аргумент; симулятор дээр бүх талбар
Буруу оролт	`driver.c` шалгалт; симулятор `readInput()`; Хүснэгт 7.1
Gantt chart UI (багшийн оноосон)	`src/web/index.html`; 6-р бүлэг
UI ↔ C холбоо	`parseC`: C-ийн бодит гаралтаас Gantt зурна; Хүснэгт 8.5